# AI Security
# - Lecture 08. Recurrent Neural Networks -

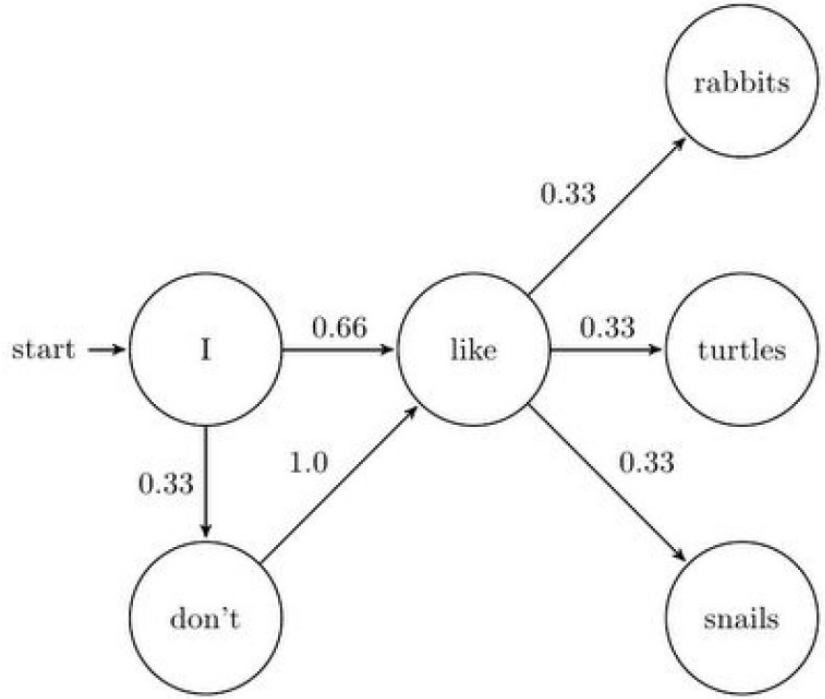
Artificial Intelligence Research (AIR) LAB
https://air.korea.ac.kr/
School of Cybersecurity
Korea University

Sangkyun Lee (이상근)

2025. Spring Semester

# Sequence Modeling

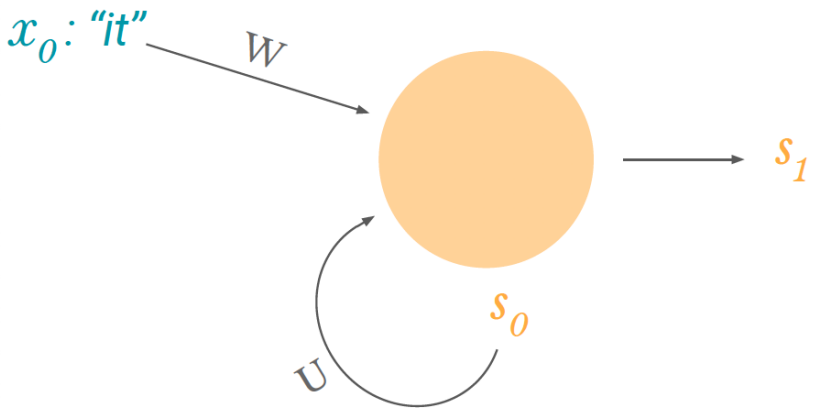
Markov models:




- Markov assumption: each state depends only on the last state
- We cannot model long-term dependencies:
  - In **France**, I had a good time and I learnt some of the _____ **language**

# RNN

RNN hidden nodes “remember” their previous state:



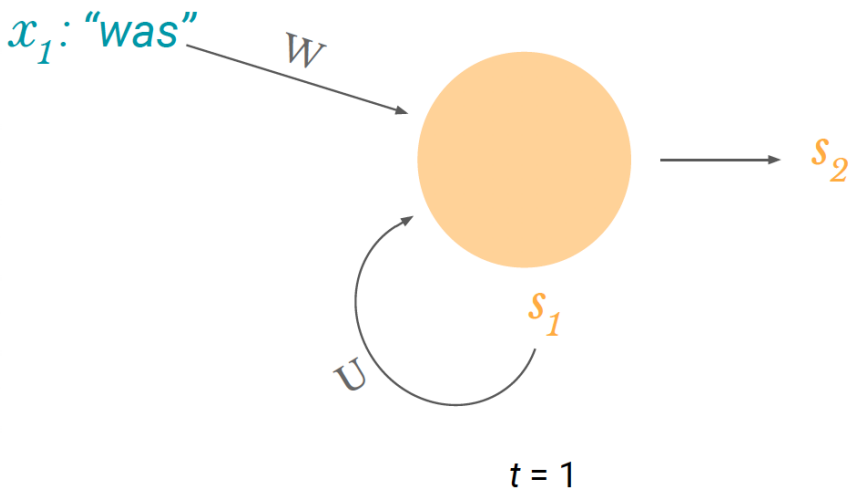

$t = 0$

$x_0$ : vector representing first word
$s_0$ : cell state at $t = 0$ (some initialization)
$s_1$ : cell state at $t = 1$

$$s_1 = tanh(Wx_0 + Us_0)$$

# RNN

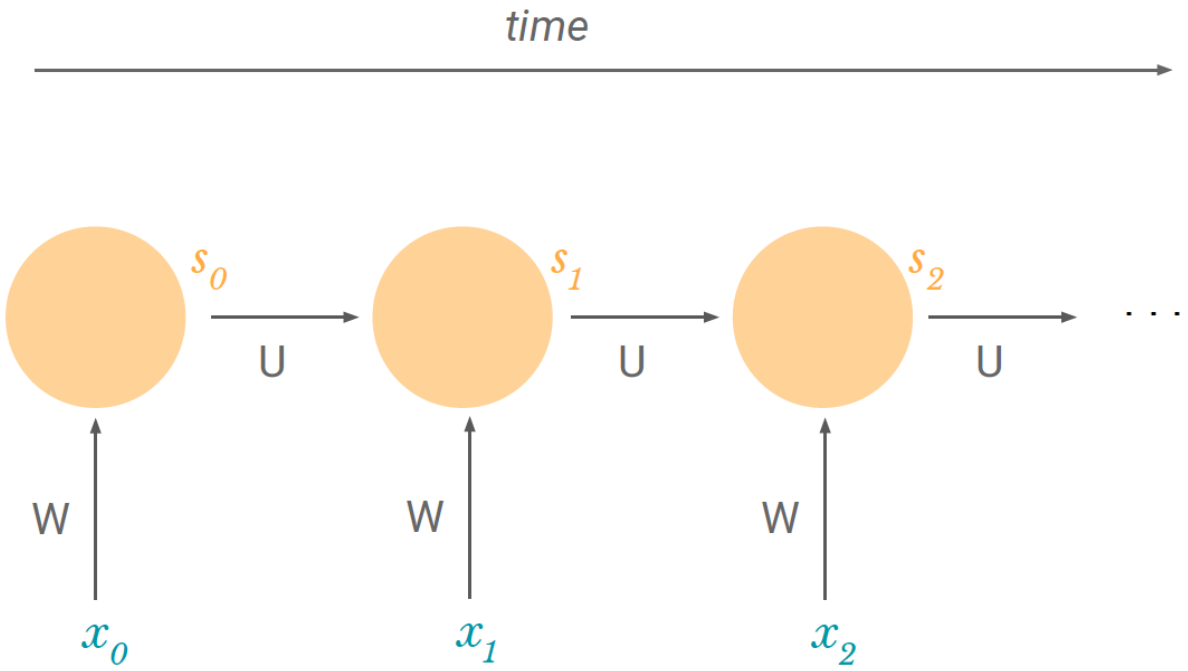
RNN hidden nodes “remember” their previous state:




$x_1$ : vector representing second word
$s_1$ : cell state at $t = 1$
$s_2$ : cell state at $t = 2$

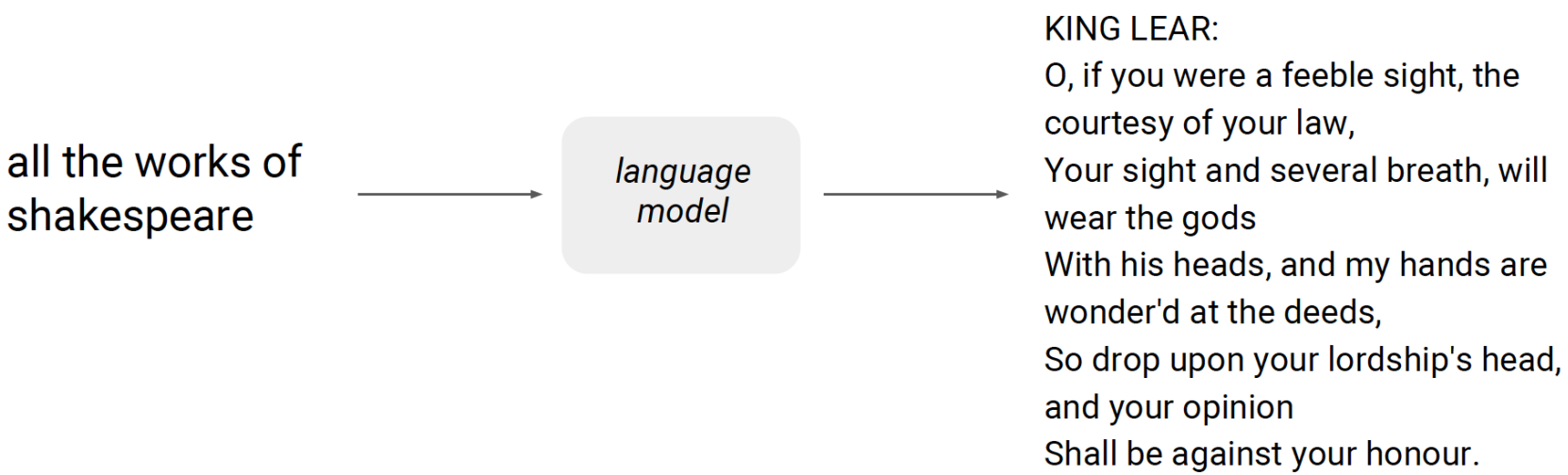
$$s_2 = tanh(Wx_1 + Us_1)$$

# Unfolding RNN Hidden States

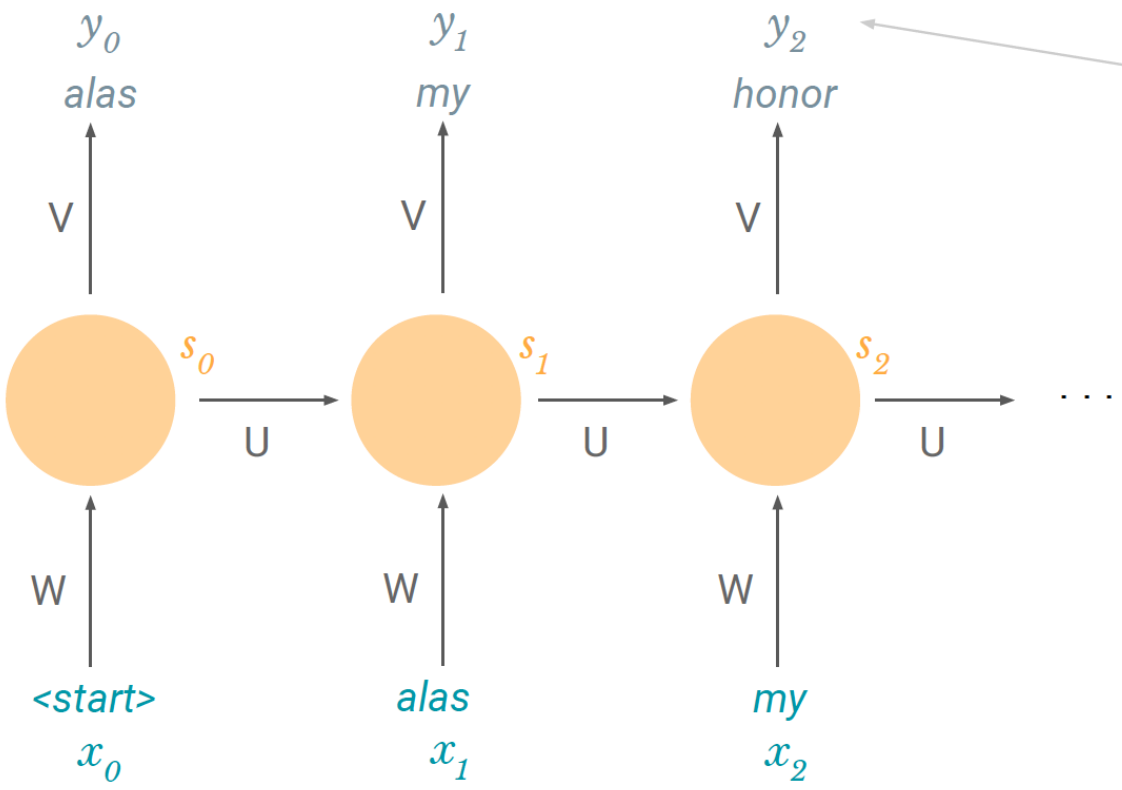



$s_n$ can contain information from all previous states

# Language Modeling



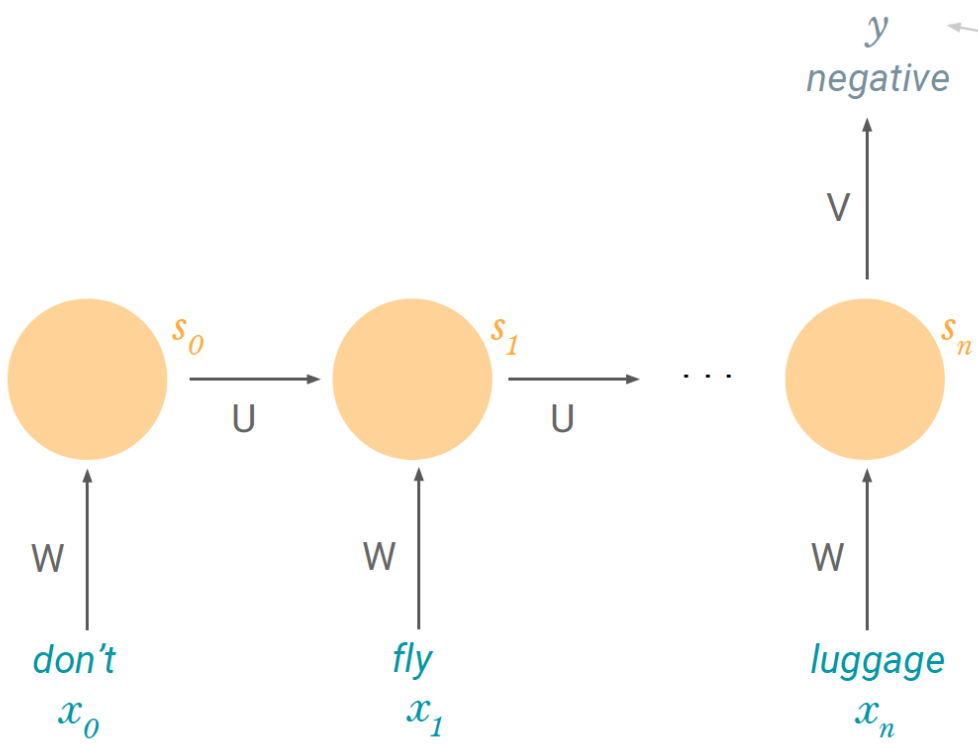
# Language Modeling




$y_i$ is actually a probability distribution over possible next words, aka a *softmax*

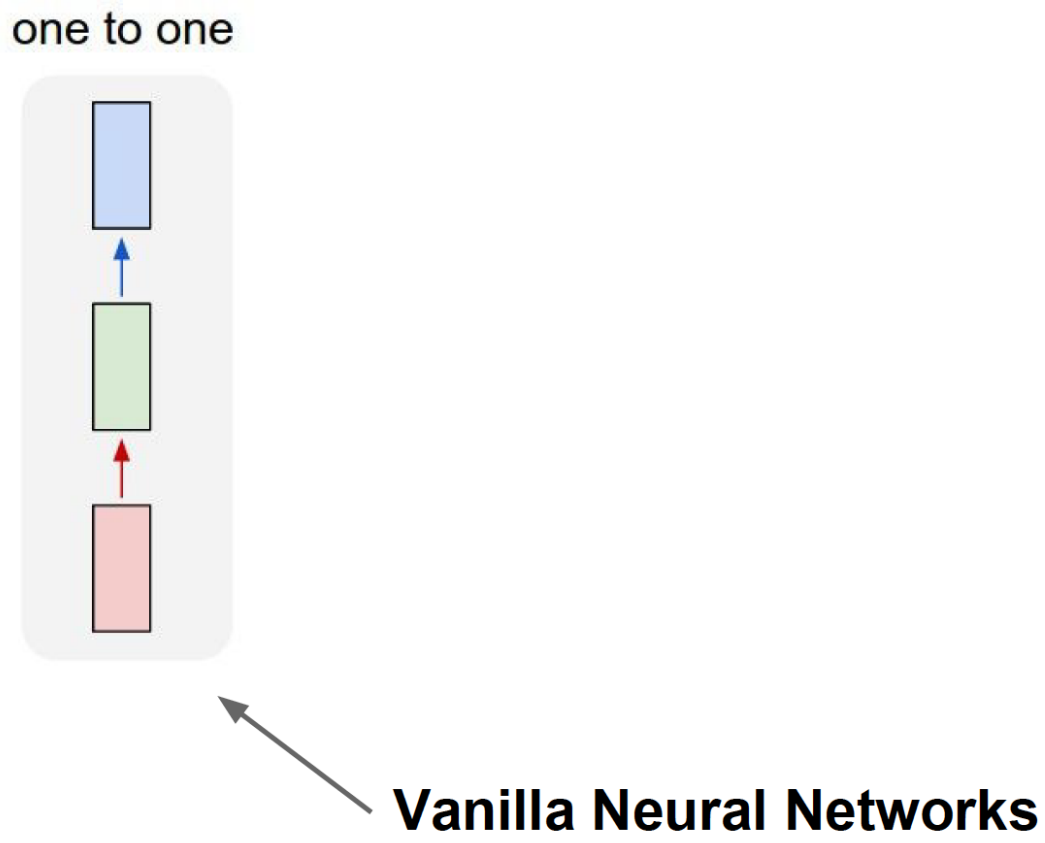
# Sentiment Analysis

# Sentiment Analysis

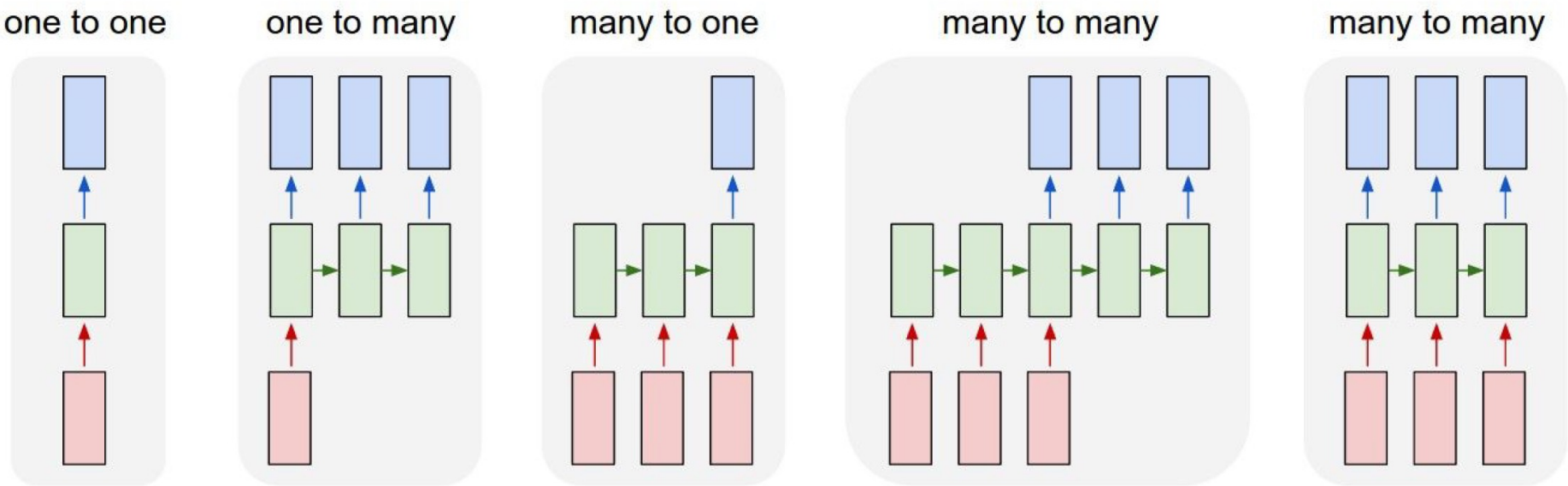



$y$ is a probability distribution over possible classes (like positive, negative, neutral), aka a *softmax*

# “Vanilla” NN

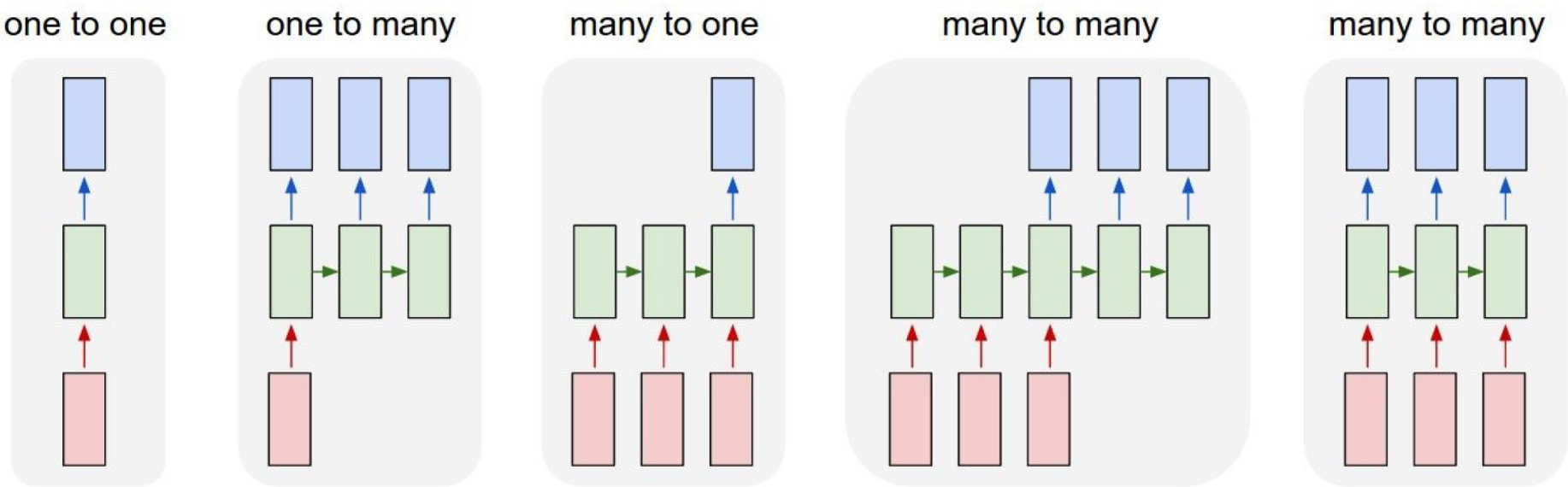


# RNN: Model Sequences



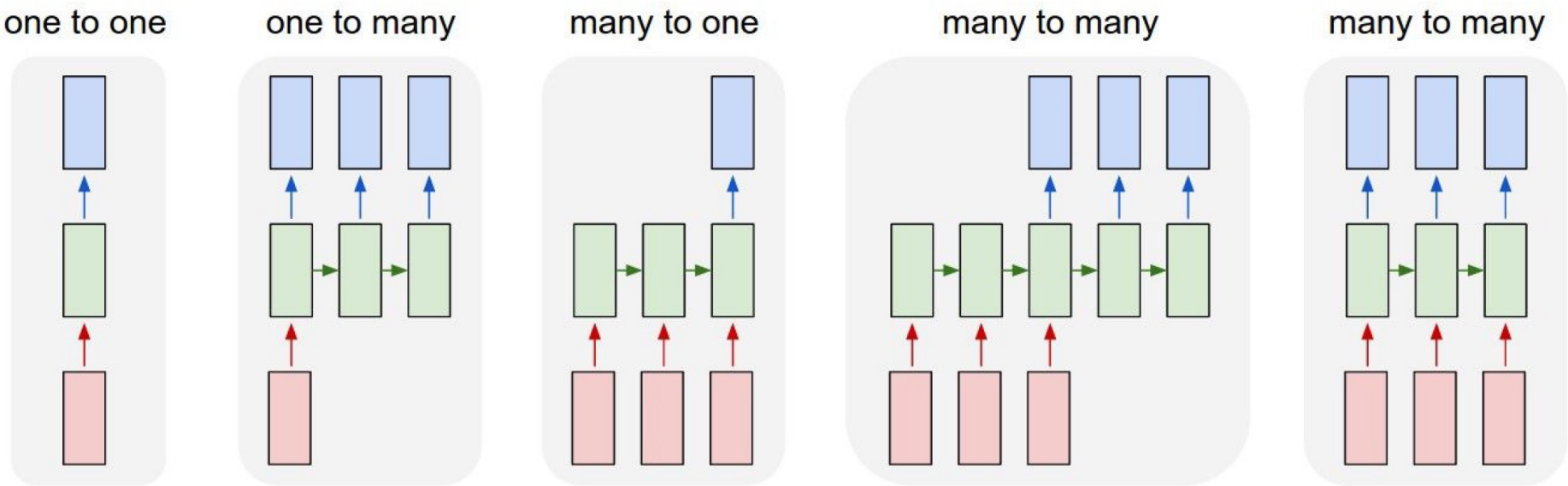

e.g. **Image Captioning**
image -> sequence of words

# RNN: Model Sequences



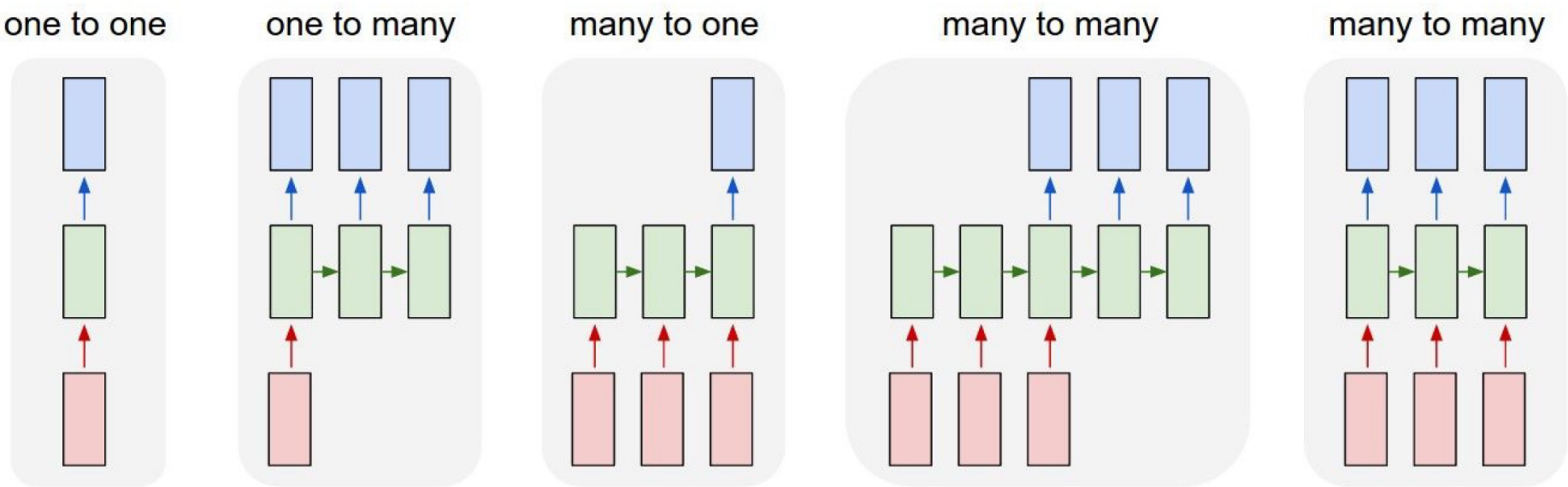

e.g. **Sentiment Classification**
sequence of words -> sentiment

# RNN: Model Sequences



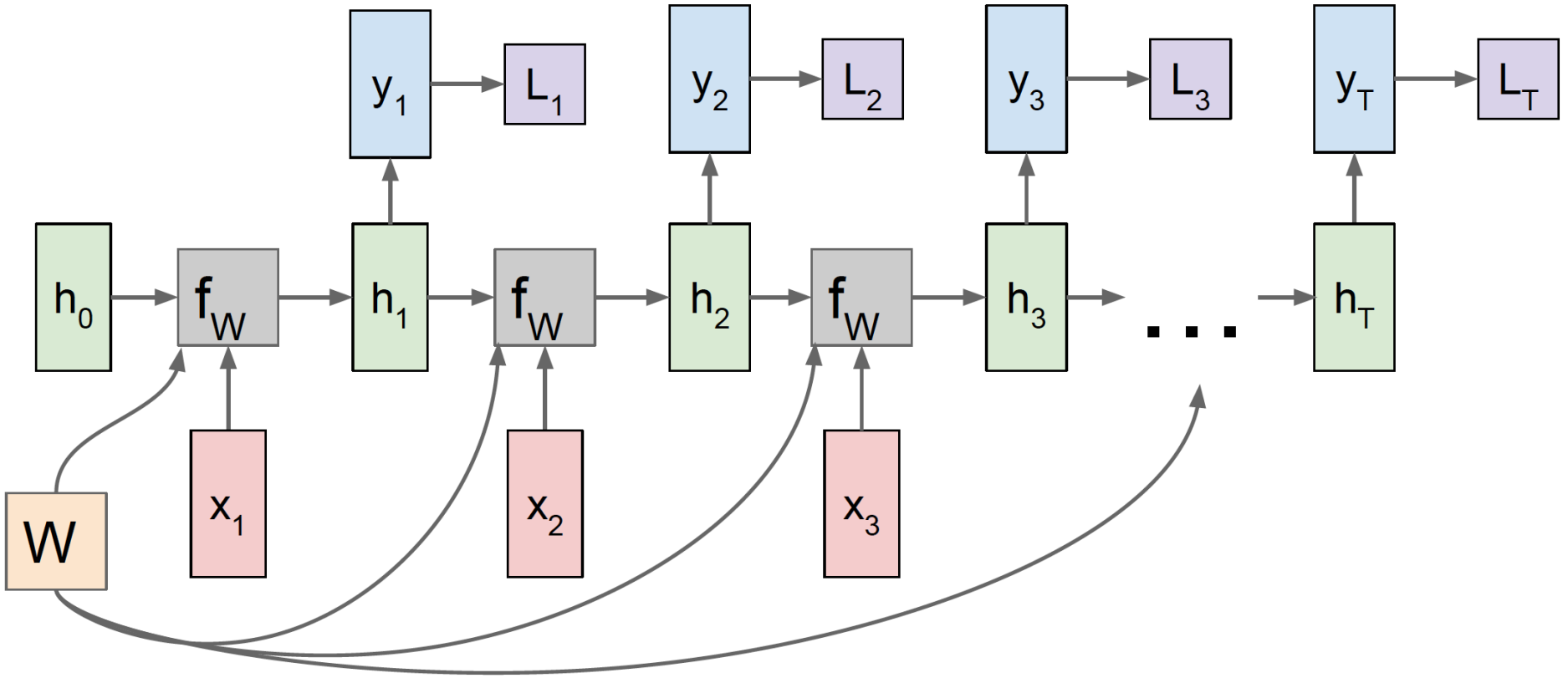

e.g. **Machine Translation**
seq of words -> seq of words

# RNN: Model Sequences




e.g. **Video classification on frame level**

# RNN Computation Graph: Many-to-Many

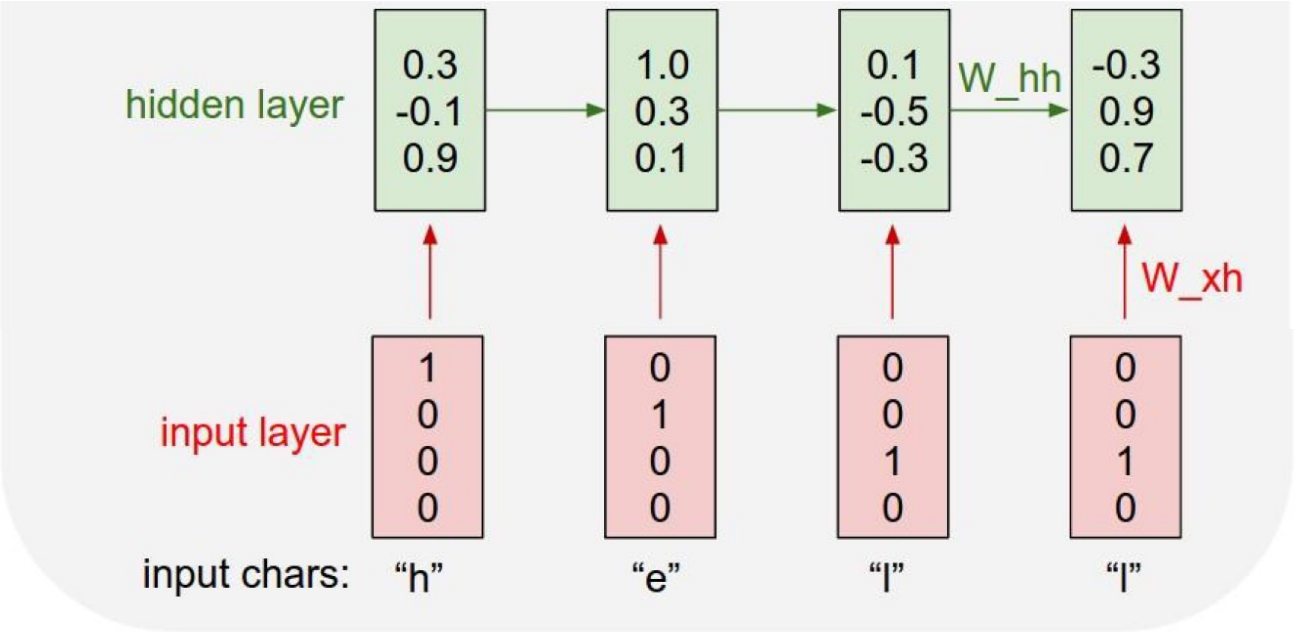


# Ex. Character-Level Language Modelling

**Vocaburary:**
**[h, e, l, o]**

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

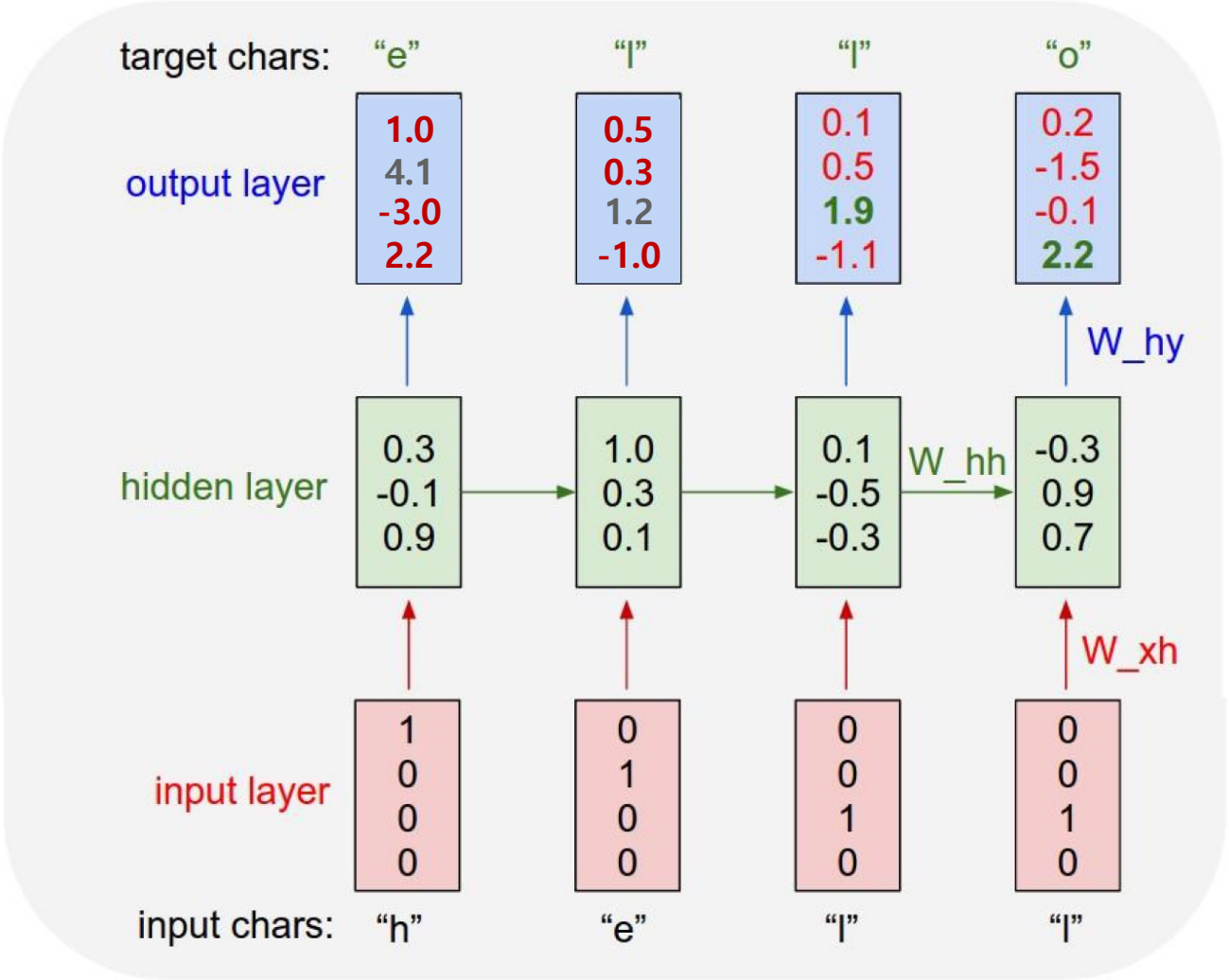
**Training seqex: “hello”**



# Ex. Character-Level Language Modelling

**Vocaburary:**
**[h, e, l, o]**

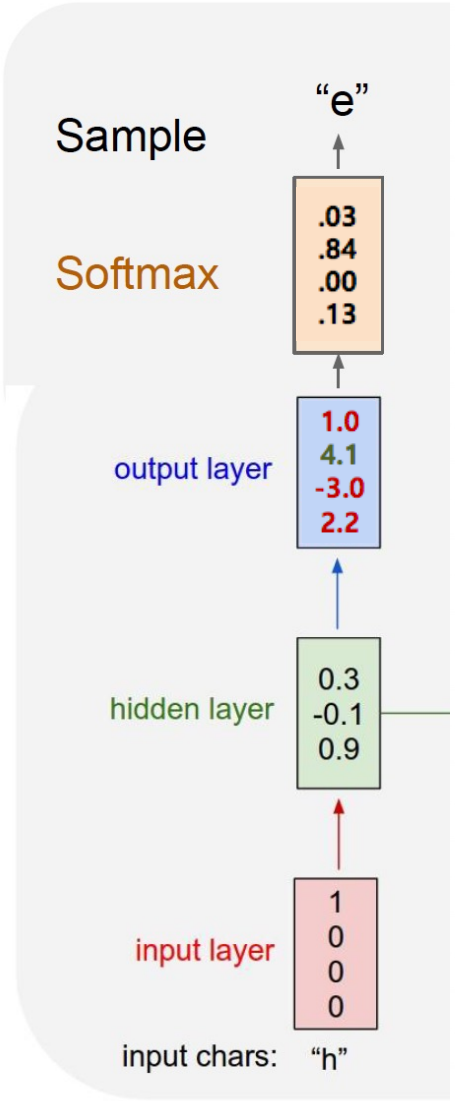
**Training sequence exa**
**“hello”**



# Ex. Character-Level Language Modelling

**Vocaburary:**
**[h, e, l, o]**

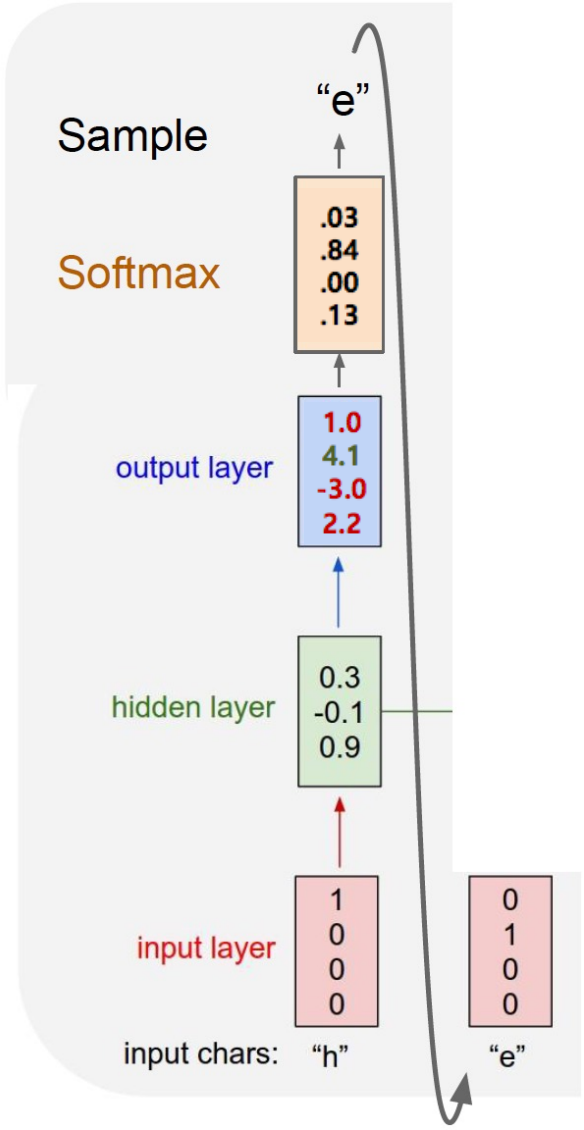
**Test-time: sample one character at a time, feeding back to the model**



# Ex. Character-Level Language Modelling

**Vocaburary:**
**[h, e, l, o]**

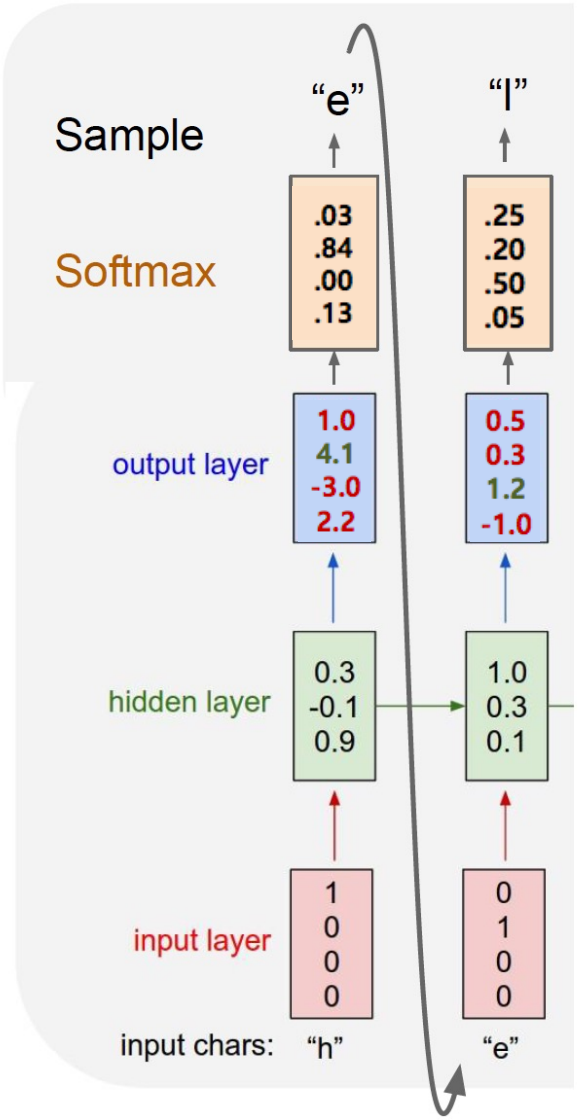
**Test-time: sample one character at a time, feeding back to the model**



# Ex. Character-Level Language Modelling

**Vocaburary:**
**[h, e, l, o]**

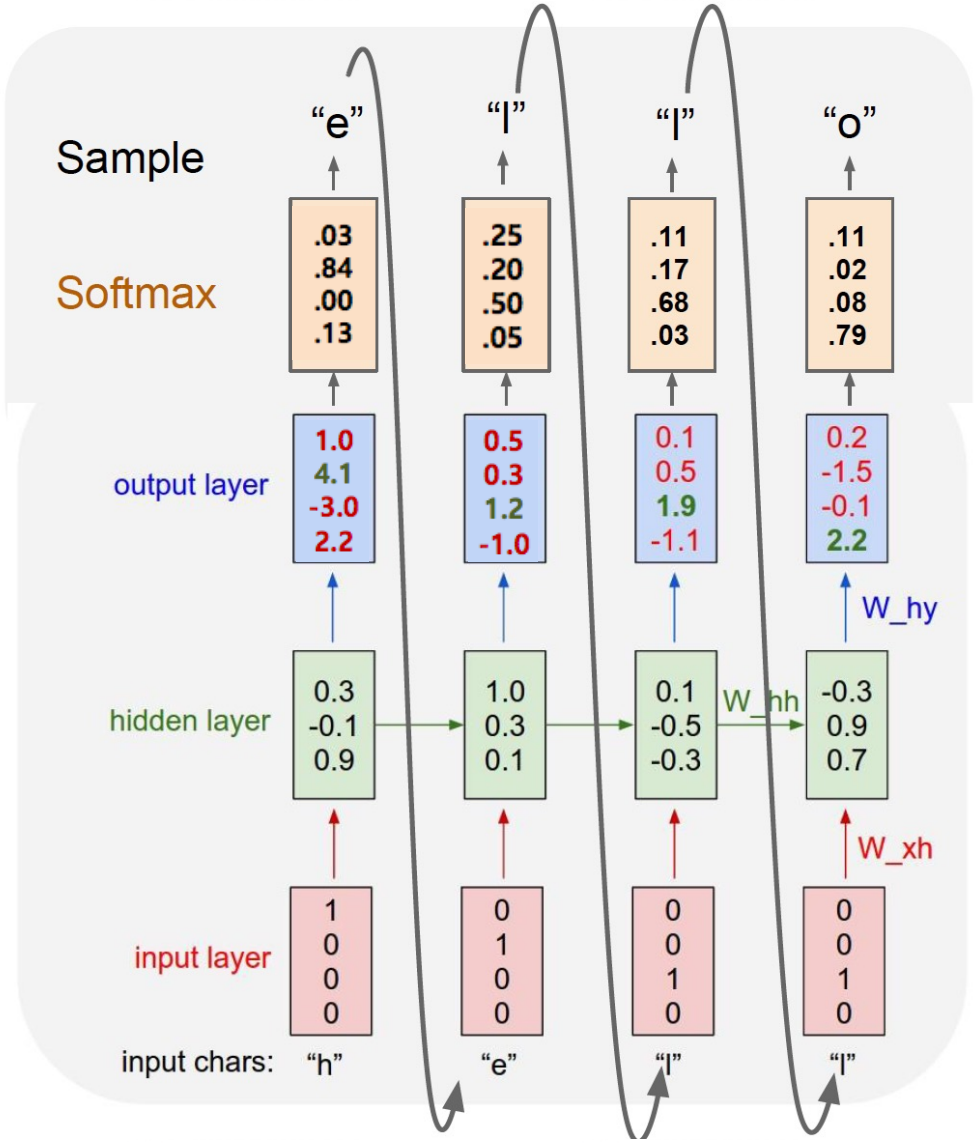
**Test-time: sample one character at a time, feeding back to the model**



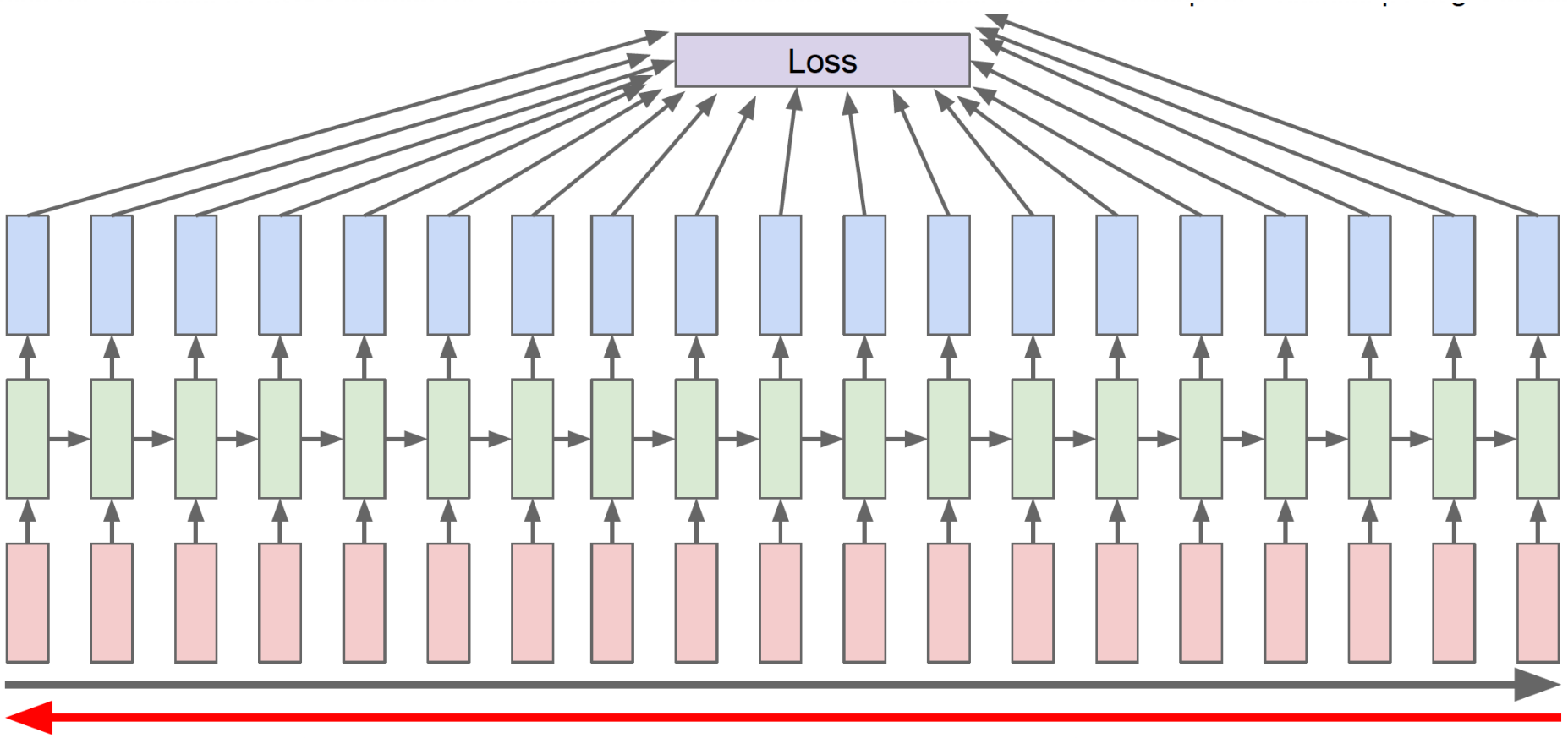
# Ex. Character-Level Language Modelling

**Vocaburary:**
**[h, e, l, o]**

**Test-time: sample one character at a time, feeding back to the model**



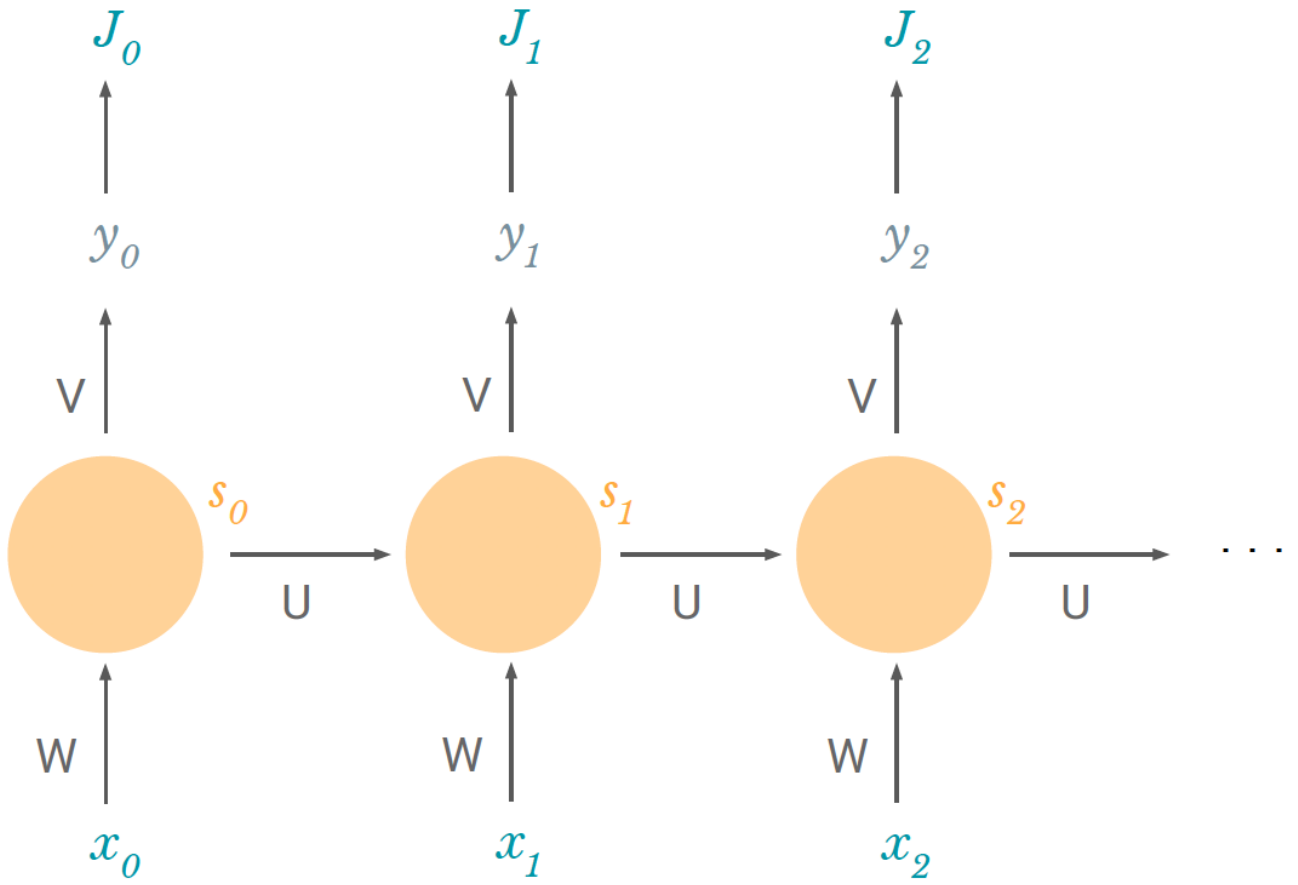
# BPTT: Back-Propagation Through Time



# Back Propagation

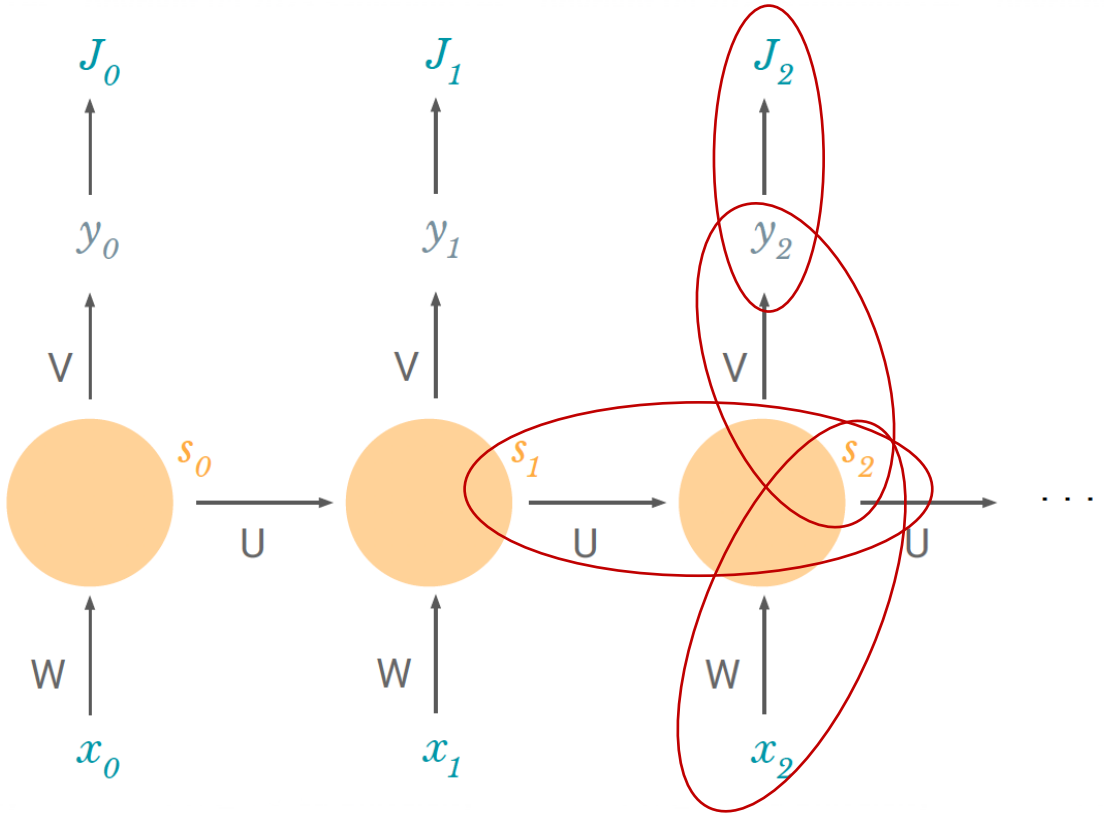
We have loss at each time step:

$$J(w) = \sum_t J_t(w)$$



# Back Propagation

$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

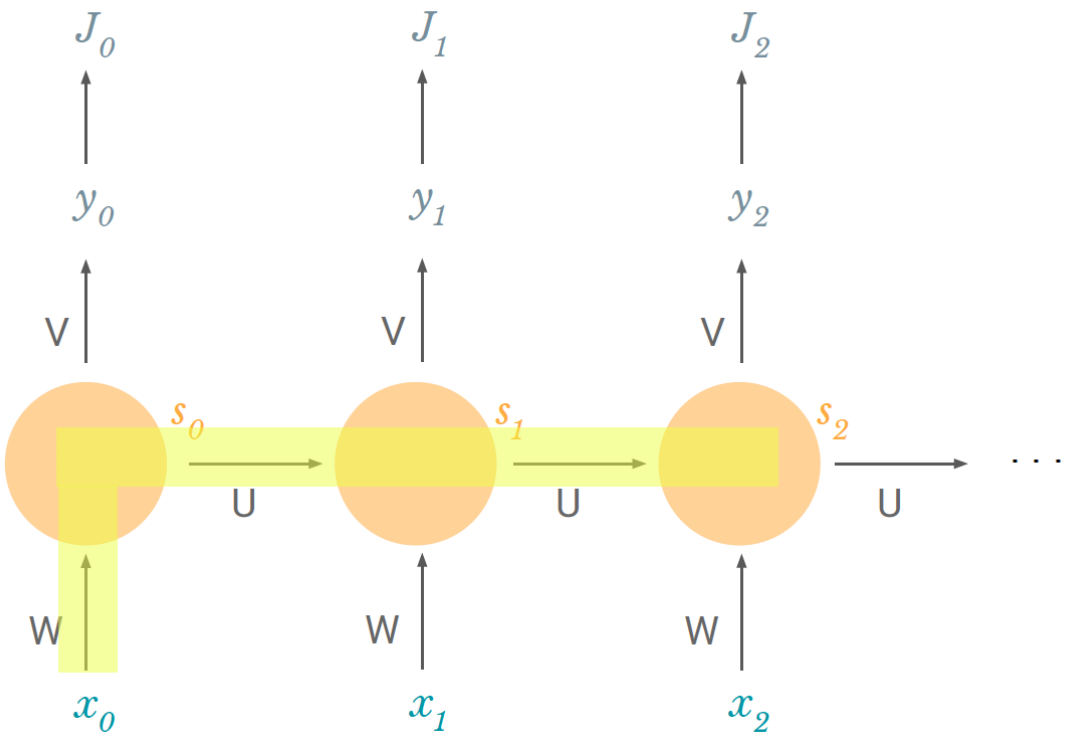



$$\frac{\partial J_2}{\partial W} =$$

$$s_2 = \tanh(Us_1 + Wx_2)$$

$s_1$ also depends on W !!

# Dependency of $s_2$ on $W$

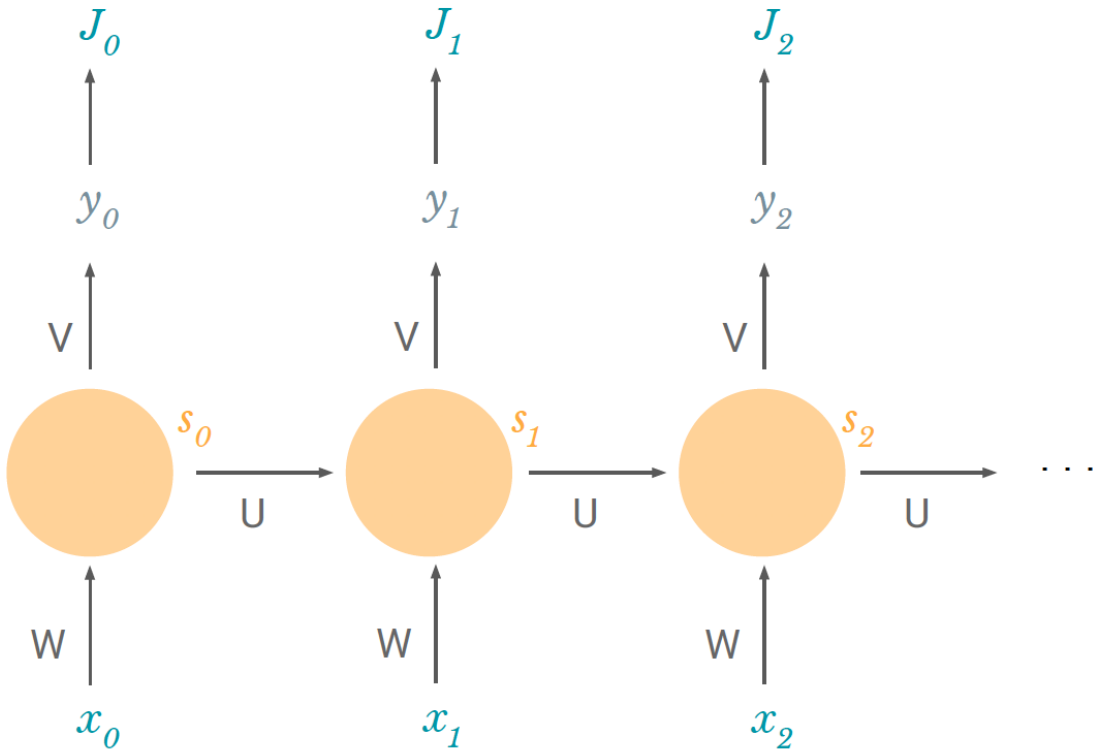



$$\frac{\partial s_2}{\partial W} + \frac{\partial s_2}{\partial s_1}\frac{\partial s_1}{\partial W} + \frac{\partial s_2}{\partial s_1}\frac{\partial s_1}{\partial s_0}\frac{\partial s_0}{\partial W}$$

# BPTT

$$\frac{\partial J_2}{\partial W} = \sum_{k=0}^{2} \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2} \boxed{\frac{\partial s_2}{\partial s_k}} \frac{\partial s_k}{\partial W}$$

$$\frac{\partial s_2}{\partial s_1} \cdots \frac{\partial s_{k+1}}{\partial s_k}$$



# Problem: Vanishing Gradient

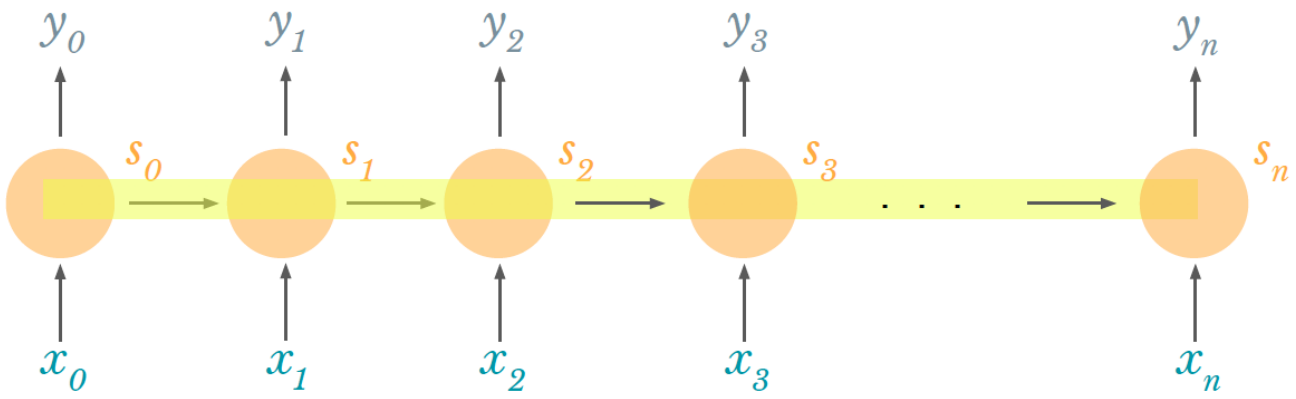
$$\frac{\partial J_n}{\partial W} = \sum_{k=0}^{n} \frac{\partial J_n}{\partial y_n} \frac{\partial y_n}{\partial s_n} \frac{\partial s_n}{\partial s_k} \frac{\partial s_k}{\partial W}$$

$$\frac{\partial s_n}{\partial s_k} \quad \frac{\partial s_n}{\partial s_{n-1}} \frac{\partial s_{n-1}}{\partial s_{n-2}} \cdots \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial s_0}$$

$$\frac{\partial s_n}{\partial s_{n-1}} \leftarrow W^T \text{diag}[\phi'(W x_j + U s_{j-1})]$$

W: sampled from *N(0,1)*

$$\phi' < 1$$




We're multiplying lots of small numbers

# Problem: Vanishing Gradient

We're multiplying lots of **small numbers**

→ Errors due to further back timesteps have increasingly **smaller gradients**

→ Parameters become biased to capture **short-term dependencies**

**Solutions:**

- Use special units instead of hidden nodes
- E.g. LSTM (Long Short-Term Memory)

# Image Captioning

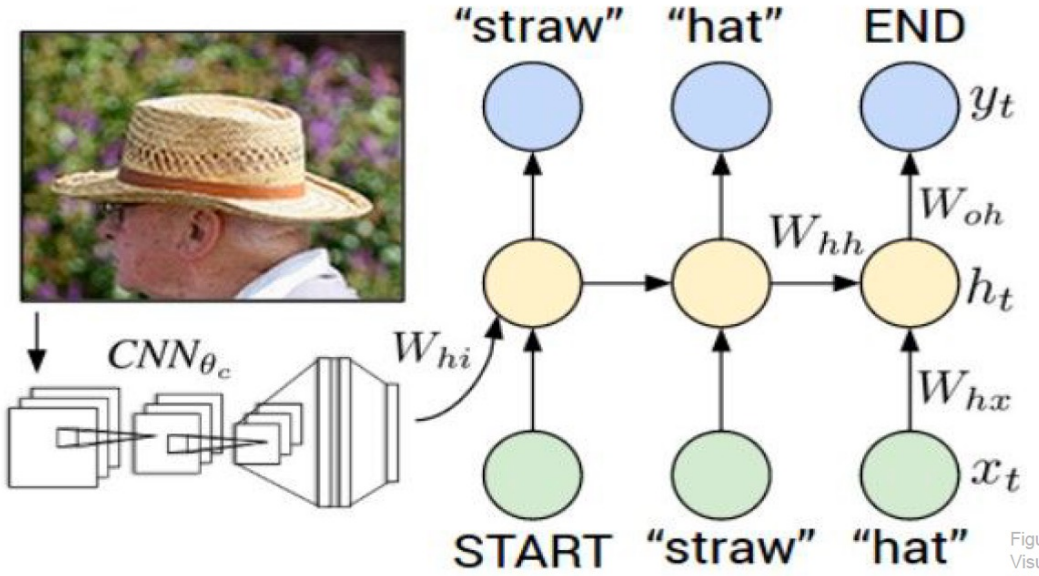
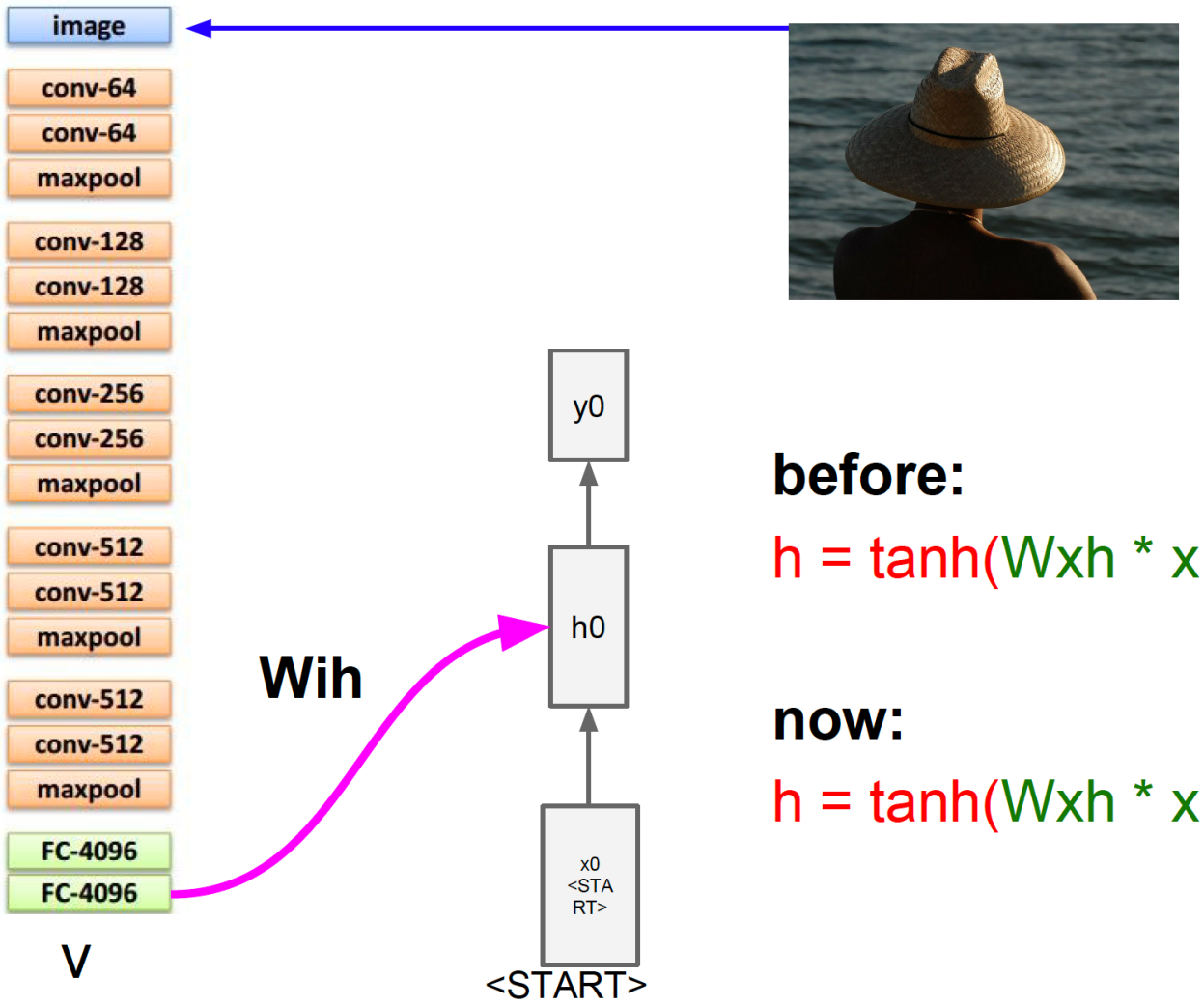



Figure from Karpathy et a, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015; figure copyright IEEE, 2015.
Reproduced for educational purposes.

Explain Images with Multimodal Recurrent Neural Networks, Mao et al.
Deep Visual-Semantic Alignments for Generating Image Descriptions, Karpathy and Fei-Fei
Show and Tell: A Neural Image Caption Generator, Vinyals et al.
Long-term Recurrent Convolutional Networks for Visual Recognition and Description, Donahue et al.
Learning a Recurrent Visual Representation for Image Caption Generation, Chen and Zitnick

# Image Captioning






test image

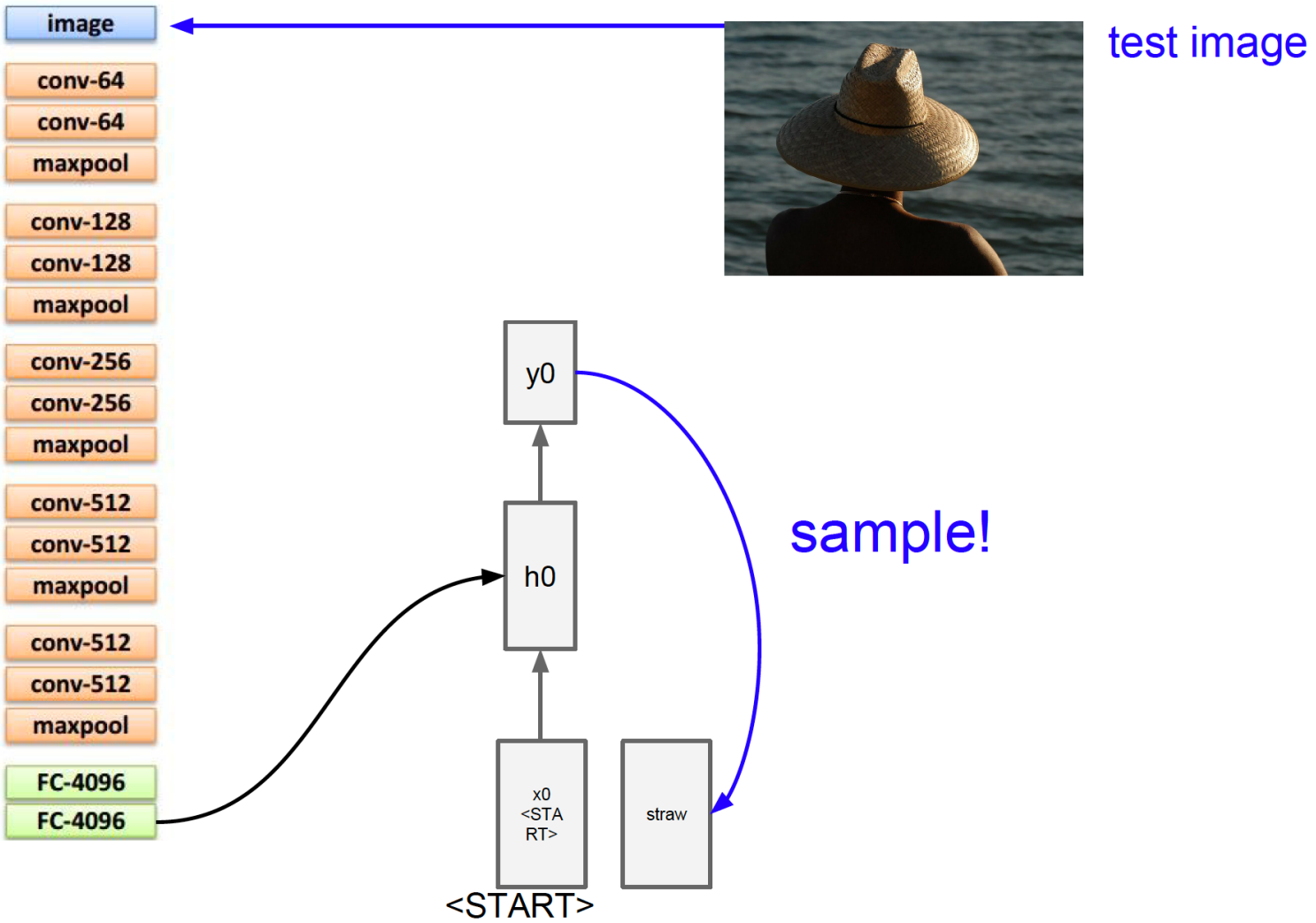
**before:**

h = tanh(Wxh * x + Whh * h)

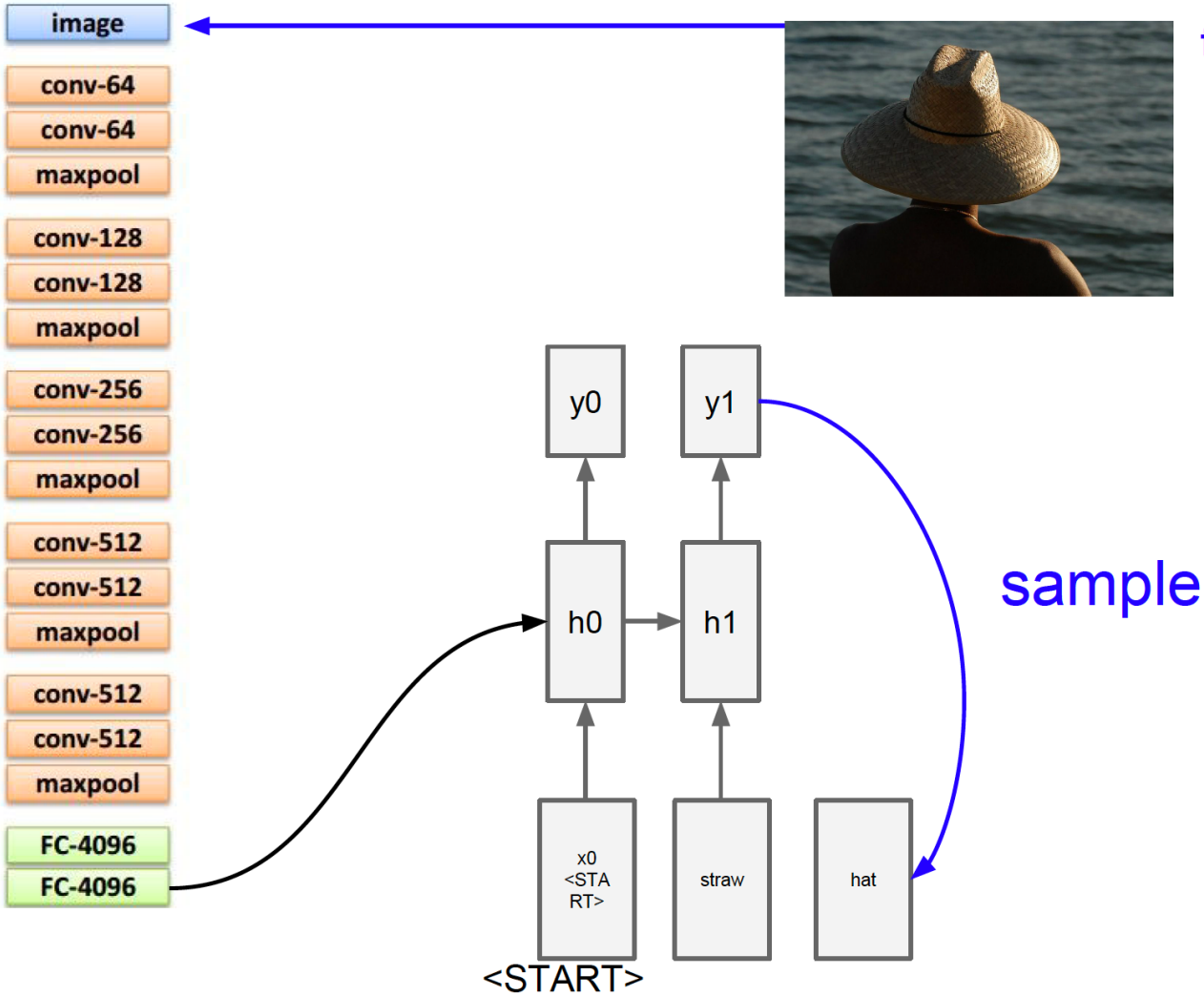
**now:**

h = tanh(Wxh * x + Whh * h + **Wih** * **v**)

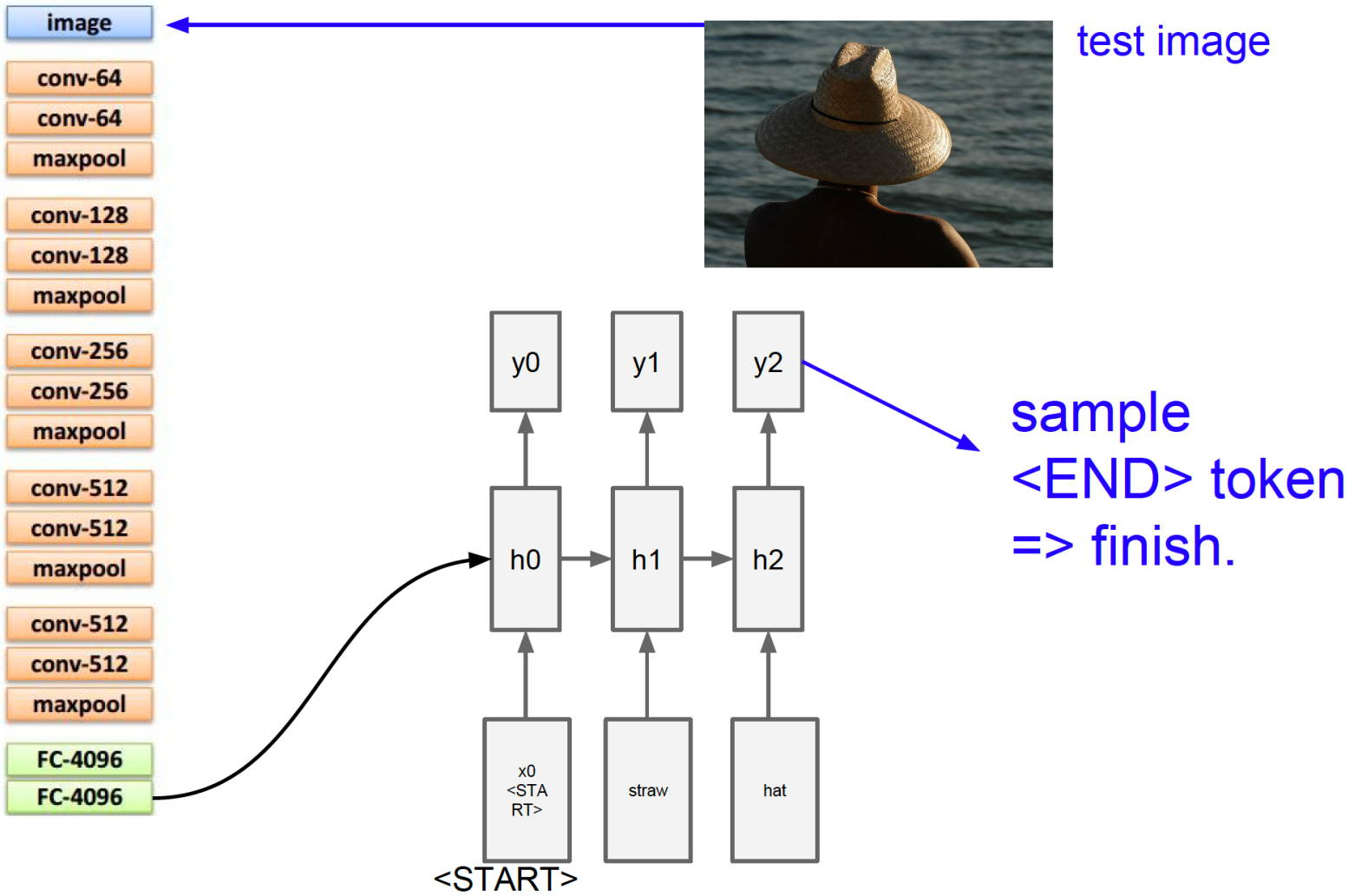
# Image Captioning



# Image Captioning



# Image Captioning



# Image Captioning: Examples

*A cat sitting on a suitcase on the floor*

*A cat is sitting on a tree branch*

*A dog is running in the grass with a frisbee*

*A white teddy bear sitting in the grass*

*Two people walking on the beach with surfboards*


*A tennis player in action on the court*

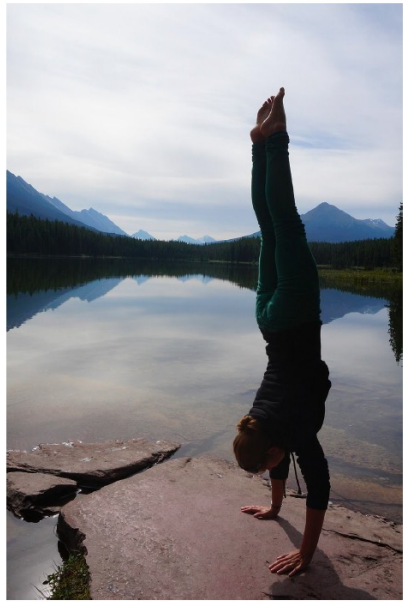
*Two giraffes standing in a grassy field*

*A man riding a dirt bike on a dirt track*

# Image Captioning: Failures

*A woman is holding a cat in her hand*

*A person holding a computer mouse on a desk*



*A woman standing on a beach holding a surfboard*

*A bird is perched on a tree branch*

*A man in a baseball uniform throwing a ball*

# Visual Question Answering


Q: What endangered animal is featured on the truck?

A: A bald eagle.
A: A sparrow.
A: A humming bird.
A: A raven.


Q: Where will the driver go if turning right?

A: Onto 24 ¾ Rd.
A: Onto 25 ¾ Rd.
A: Onto 23 ¾ Rd.
A: Onto Main Street.

Q: When was the picture taken?

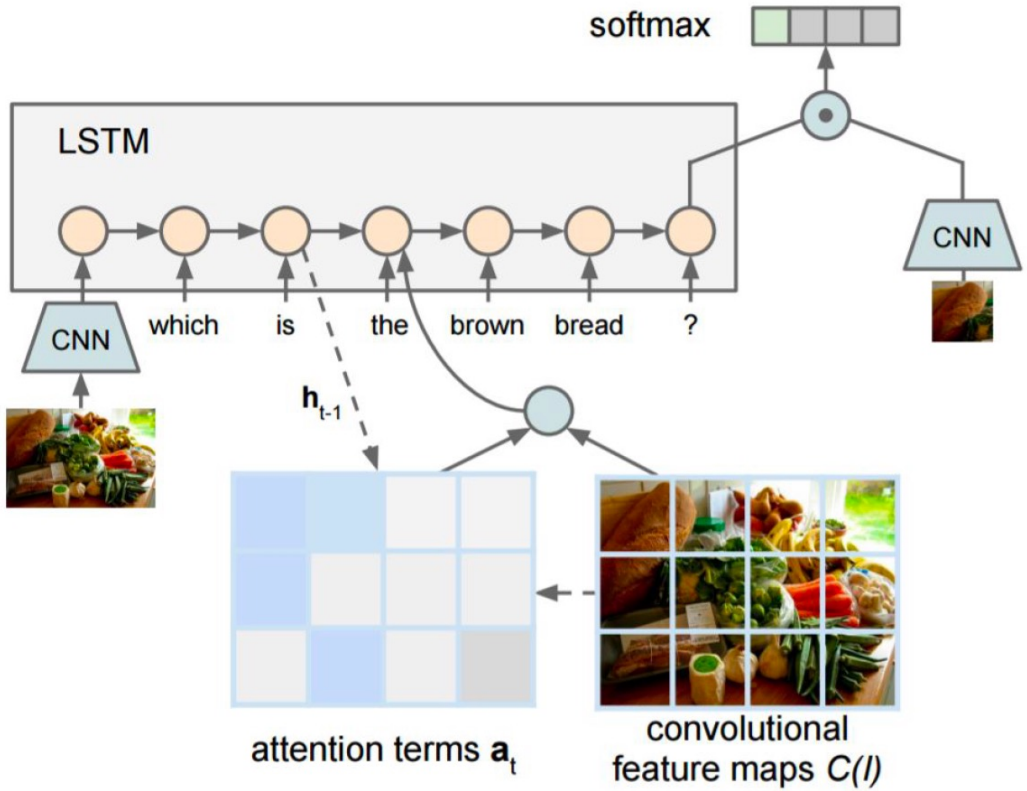
A: During a wedding.
A: During a bar mitzvah.
A: During a funeral.
A: During a Sunday church service

Q: Who is under the umbrella?

A: Two women.
A: A child.
A: An old man.
A: A husband and a wife.

Agrawal et al, "VQA: Visual Question Answering", ICCV 2015
Zhu et al, "Visual 7W: Grounded Question Answering in Images", CVPR 2016
Figure from Zhu et al, copyright IEEE 2016. Reproduced for educational purposes.

# Visual QA: RNN + Attention




Zhu et al, "Visual 7W: Grounded Question Answering in Images", CVPR 2016




What kind of animal is in the photo?

A **cat**.


Why is the person holding a knife?

To cut the **cake** with.

# Long Short Term Memory

## Multilayer RNNs

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$h \in \mathbb{R}^n$ $W^l$ $[n \times 2n]$

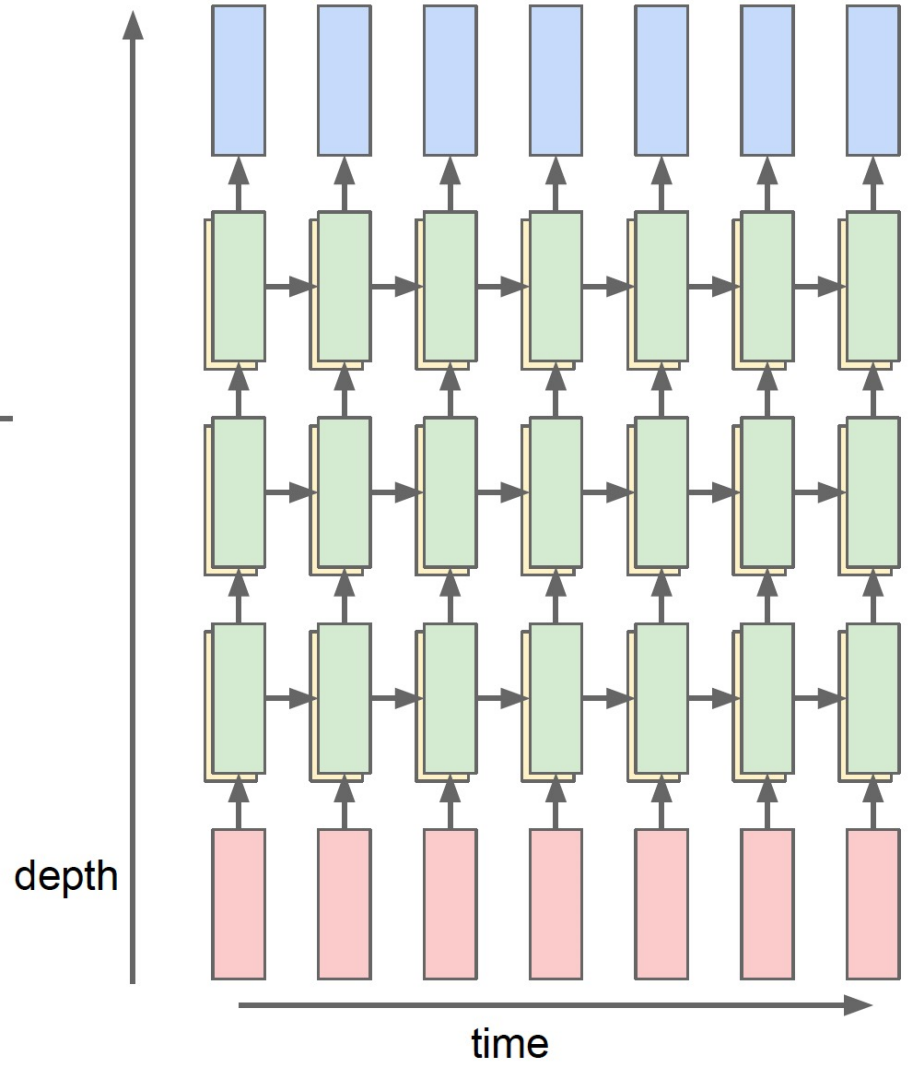
## LSTM:

$W^l$ $[4n \times 2n]$

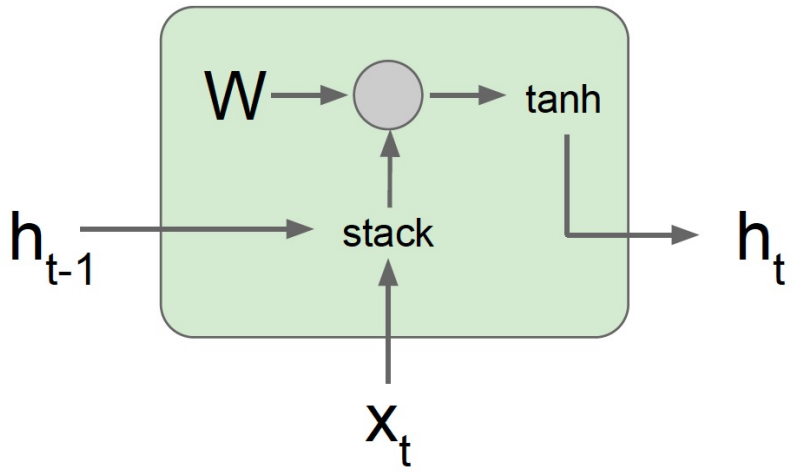
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \tanh \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

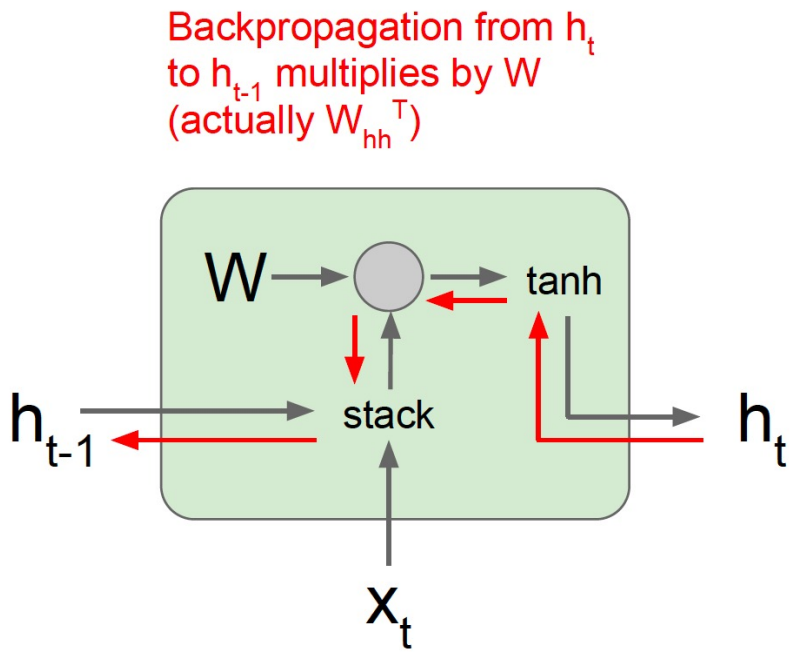


# Vanilla RNN Gradient Flow



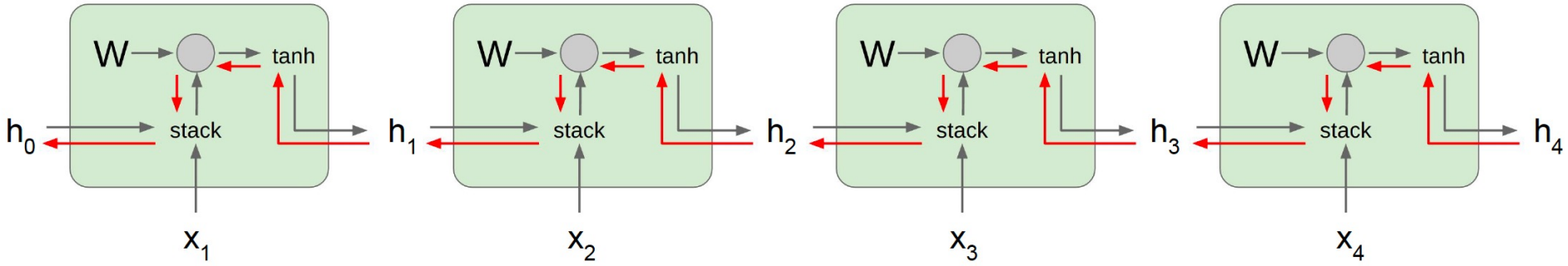

$$
\begin{aligned}
h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\
&= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\
&= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)
\end{aligned}
$$

# Vanilla RNN Gradient Flow




$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$
$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

# Vanilla RNN Gradient Flow




Computing gradient of $h_0$ involves many factors of W (and repeated tanh)

Largest singular value > 1:
**Exploding gradients**

Largest singular value < 1:
**Vanishing gradients**

# LSTM [Hochreiter et al., 1997]

**Vanilla RNN**

$$h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

**LSTM**

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

# LSTM [Hochreiter et al., 1997]

$$h_t = \tanh\left(\begin{bmatrix} W_{hh} & W_{xh} \end{bmatrix} \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix}\right)$$

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} \begin{bmatrix} W_{hh}^i & W_{xh}^i \\ W_{hh}^f & W_{xh}^f \\ W_{hh}^o & W_{xh}^o \\ W_{hh}^g & W_{xh}^g \end{bmatrix} \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix}$$

$$h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

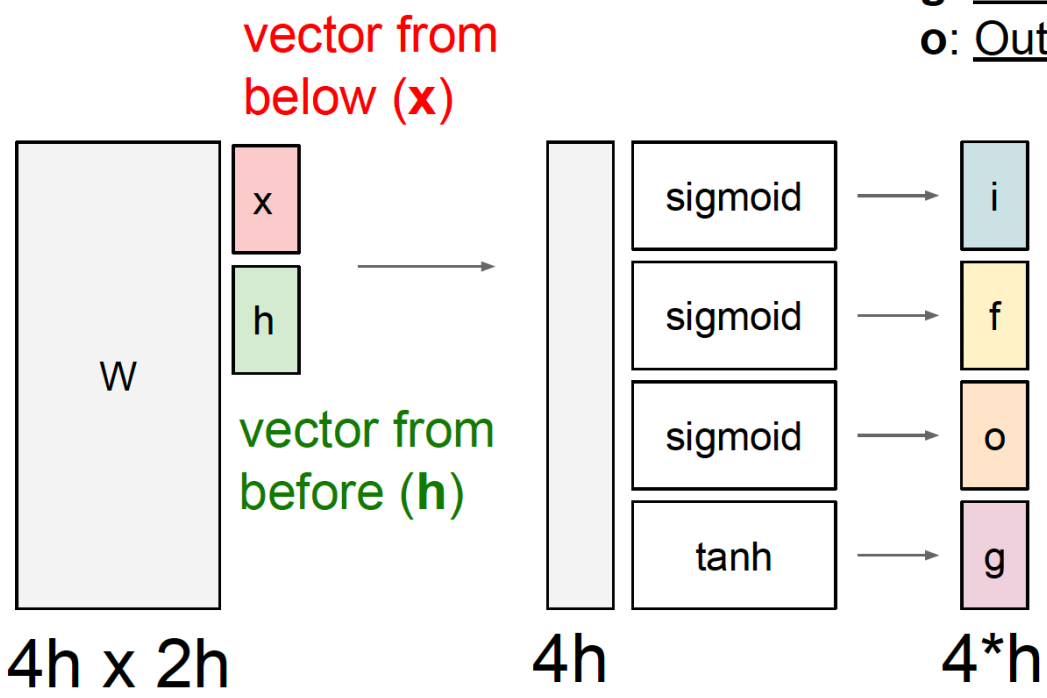
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

# LSTM [Hochreiter et al., 1997]

**f**: Forget gate, Whether to erase cell
**i**: Input gate, whether to write to cell
**g**: Gate gate (?), How much to write to cell
**o**: Output gate, How much to reveal cell

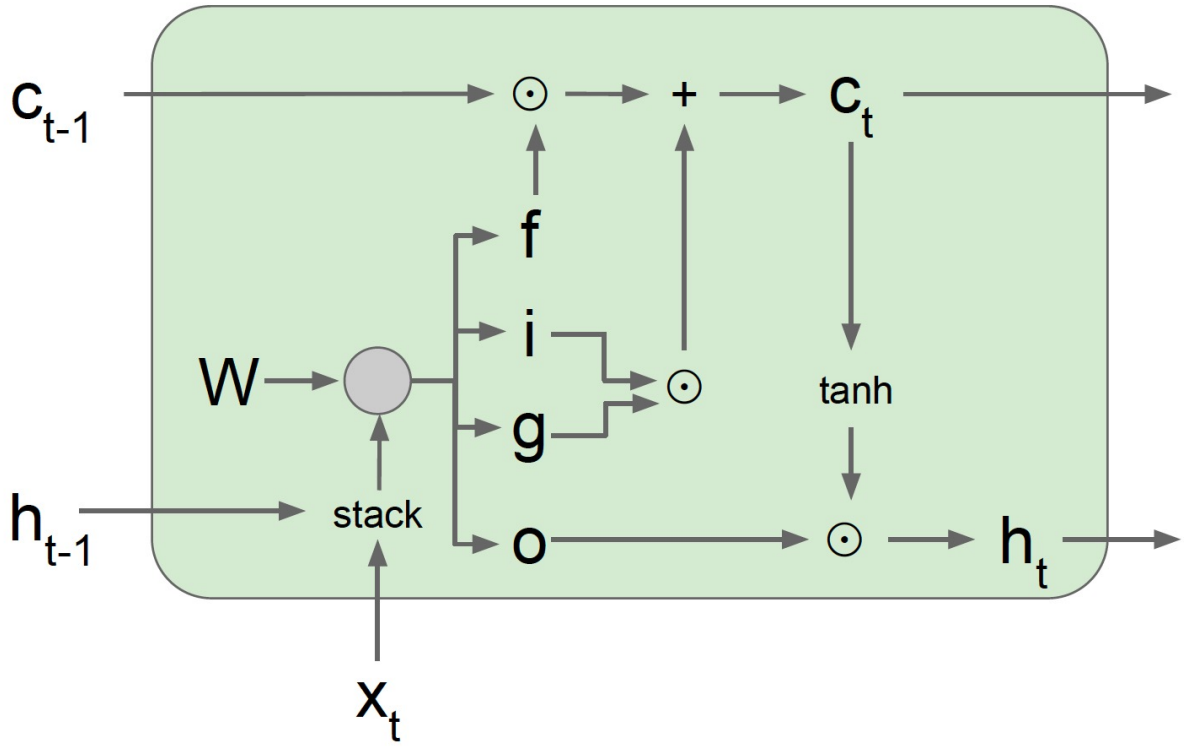



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

# LSTM: Gradient Flow

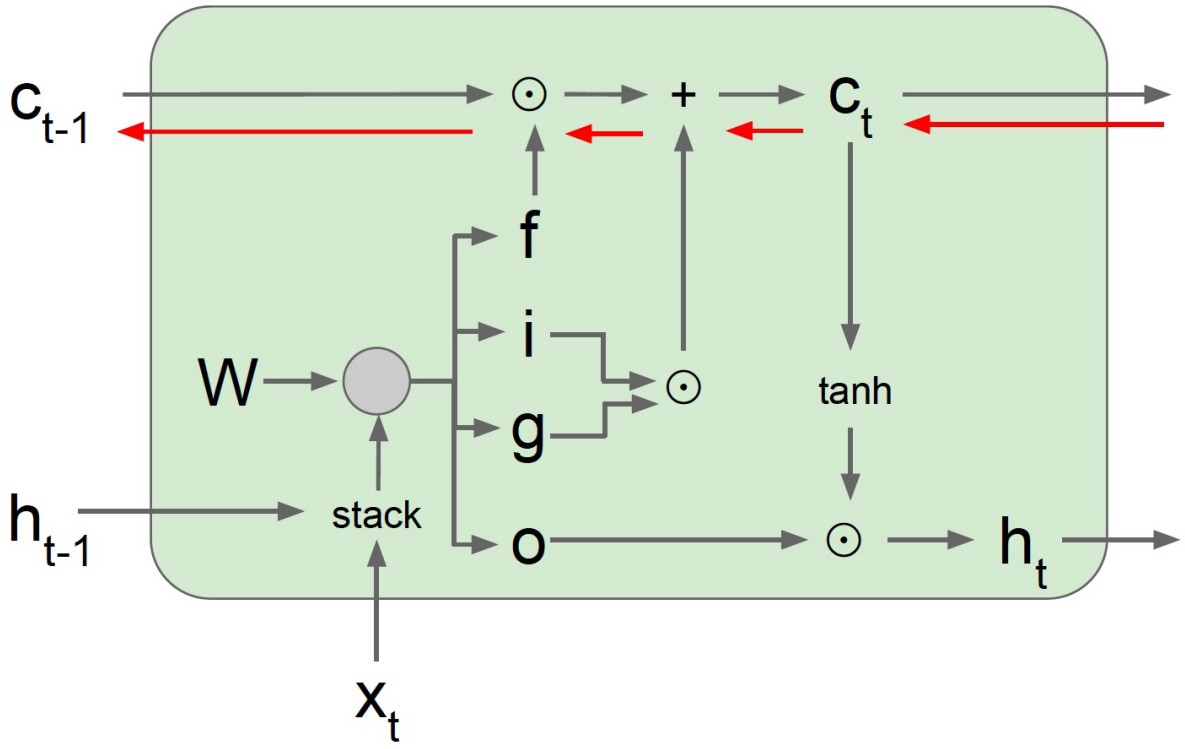



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

# LSTM: Gradient Flow



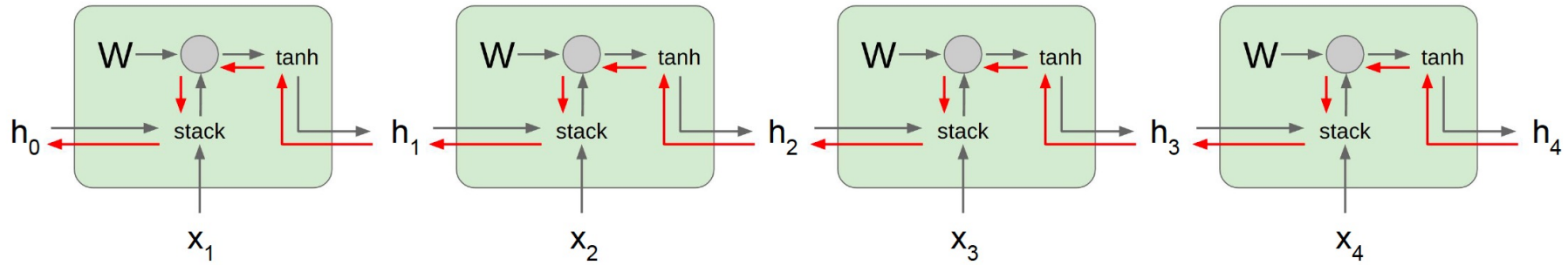

Backpropagation from $c_t$ to $c_{t-1}$ only elementwise multiplication by f, no matrix multiply by W

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

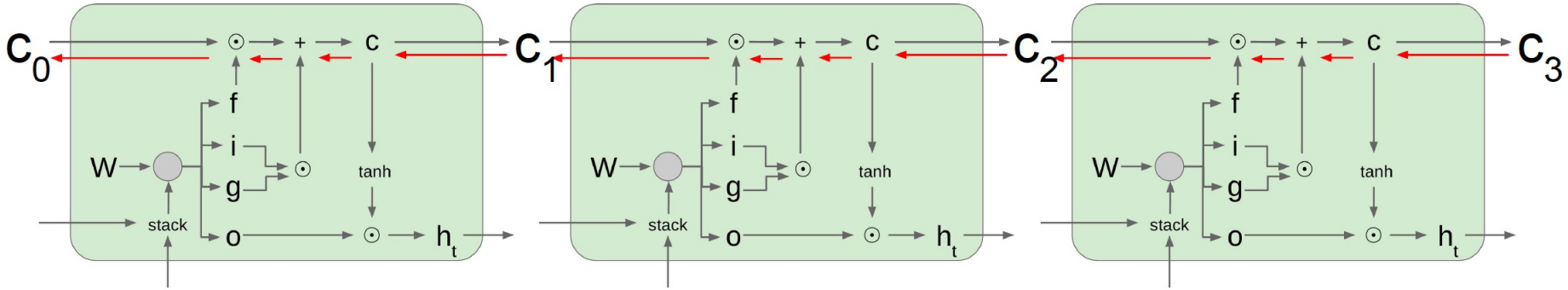
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

# Vanilla vs. LSTM: Gradient Flow




Uninterrupted gradient flow!

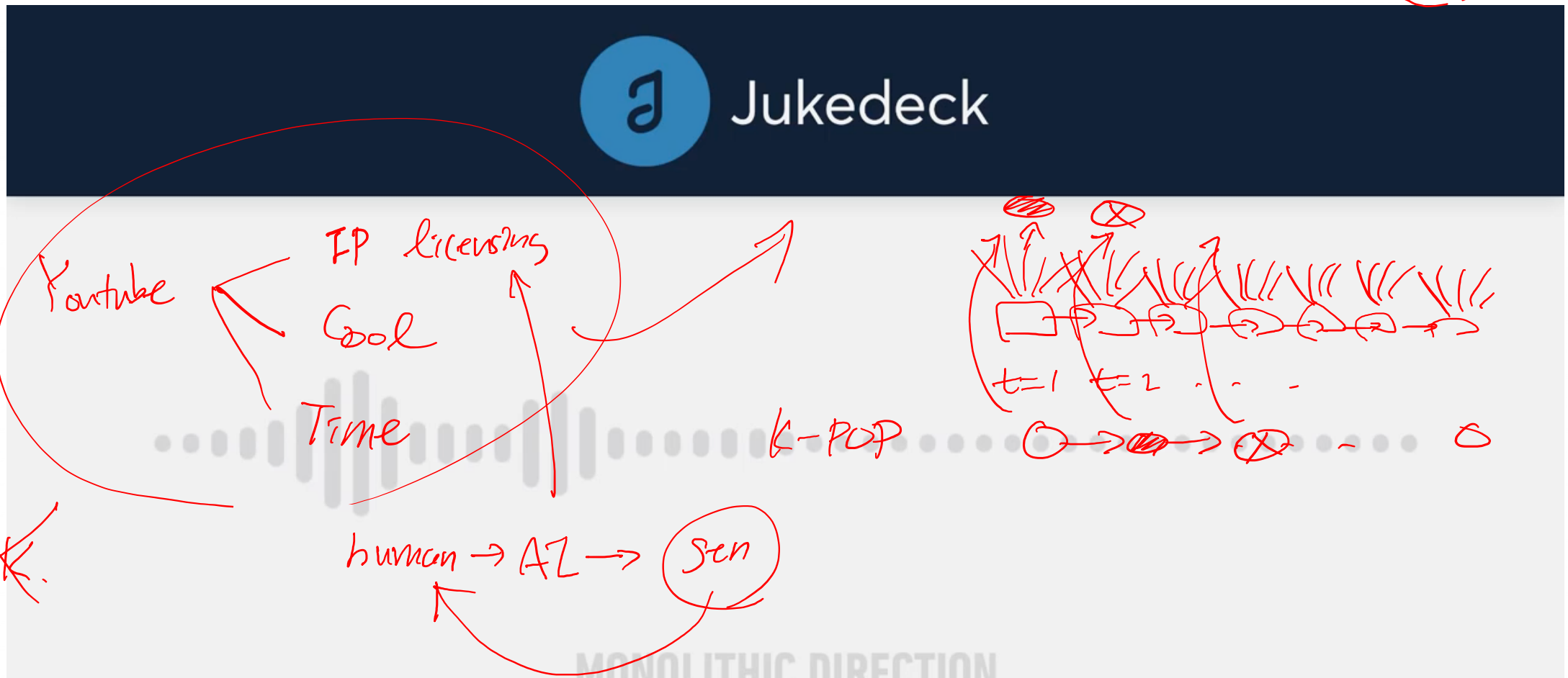


# Gated Recurrent Unit

**GRU** [*Learning phrase representations using rnn encoder-decoder for statistical machine translation*, Cho et al. 2014]

$$r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r)$$
$$z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z)$$
$$\tilde{h}_t = \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h)$$
$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

# Jukedeck



# Thank You